DATA MINING COURSE READINGS

Chapter 07: Association Analysis · Basic & Advanced Conce

Curated Textbook Readings from Tan et al. · Matching Lecture: IE500_DM_07_Association_Analysis.pdf

REFERENCE TEXTBOOK

Introduction to Data Mining (2nd Edition, Global Edition)

Authors: Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, Vipin Kumar | Pearson, 2019

Curated by: Ehsan Shahbazi | Shahid Beheshti University (SBU)

INCLUDED CHAPTER SECTIONS & TOPIC SCOPE

- **Sections 4.1·4.4: Association Rules & Apriori** — Book p. 215·245
  Itemsets, support, confidence, the Apriori principle, candidate generation/pruning, rule generation
- **Section 4.7: Pattern Evaluation & Interestingness** — Book p. 258·274
  Limitations of Support-Confidence framework, Lift, interest factor, correlation measures
- **Sections 7.1·7.3: Advanced Association Mining** — Book p. 560·572
  Handling categorical attributes, handling continuous attributes (discretization), concept hierarchies

STUDENT STUDY GUIDE & SCOPE

Covers frequent itemset mining, the Apriori algorithm and anti-monotonicity pruning, rule generation, pattern interestingness (Lift vs Confidence), and handling continuous/categorical attributes in association rules.

4

Association Analysis: Basic Concepts and Algorithms

Many business enterprises accumulate large quantities of data from their day-to-day operations. For example, huge amounts of customer purchase data are collected daily at the checkout counters of grocery stores. Table 4.1 gives an example of such data, commonly known as **market basket transactions**. Each row in this table corresponds to a transaction, which contains a unique identifier labeled TID and a set of items bought by a given customer. Retailers are interested in analyzing the data to learn about the purchasing behavior of their customers. Such valuable information can be used to support a variety of business-related applications such as marketing promotions, inventory management, and customer relationship management.

This chapter presents a methodology known as **association analysis**, which is useful for discovering interesting relationships hidden in large data sets. The uncovered relationships can be represented in the form of sets of items present in many transactions, which are known as **frequent itemsets**,

Table 4.1. An example of market basket transactions.

TID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

or **association rules**, that represent relationships between two itemsets. For example, the following rule can be extracted from the data set shown in Table 4.1:

$$\{\texttt{Diapers}\} \longrightarrow \{\texttt{Beer}\}.$$

The rule suggests a relationship between the sale of diapers and beer because many customers who buy diapers also buy beer. Retailers can use these types of rules to help them identify new opportunities for cross-selling their products to the customers.

Besides market basket data, association analysis is also applicable to data from other application domains such as bioinformatics, medical diagnosis, web mining, and scientific data analysis. In the analysis of Earth science data, for example, association patterns may reveal interesting connections among the ocean, land, and atmospheric processes. Such information may help Earth scientists develop a better understanding of how the different elements of the Earth system interact with each other. Even though the techniques presented here are generally applicable to a wider variety of data sets, for illustrative purposes, our discussion will focus mainly on market basket data.

There are two key issues that need to be addressed when applying association analysis to market basket data. First, discovering patterns from a large transaction data set can be computationally expensive. Second, some of the discovered patterns may be spurious (happen simply by chance) and even for non-spurious patterns, some are more interesting than others. The remainder of this chapter is organized around these two issues. The first part of the chapter is devoted to explaining the basic concepts of association analysis and the algorithms used to efficiently mine such patterns. The second part of the chapter deals with the issue of evaluating the discovered patterns in order to help prevent the generation of spurious results and to rank the patterns in terms of some interestingness measure.

4.1 Preliminaries

This section reviews the basic terminology used in association analysis and presents a formal description of the task.

Binary Representation Market basket data can be represented in a binary format as shown in Table 4.2, where each row corresponds to a transaction and each column corresponds to an item. An item can be treated as a binary variable whose value is one if the item is present in a transaction and zero otherwise. Because the presence of an item in a transaction is often considered

Table 4.2. A binary $0/1$ representation of market basket data.

TID	Bread	Milk	Diapers	Beer	Eggs	Cola
1	1	1	0	0	0	0
2	1	0	1	1	1	0
3	0	1	1	1	0	1
4	1	1	1	1	0	0
5	1	1	1	0	0	1

more important than its absence, an item is an **asymmetric** binary variable. This representation is a simplistic view of real market basket data because it ignores important aspects of the data such as the quantity of items sold or the price paid to purchase them. Methods for handling such non-binary data will be explained in Chapter 7.

Itemset and Support Count Let $I = \{i_1, i_2, \ldots, i_d\}$ be the set of all items in a market basket data and $T = \{t_1, t_2, \ldots, t_N\}$ be the set of all transactions. Each transaction t_i contains a subset of items chosen from I. In association analysis, a collection of zero or more items is termed an itemset. If an itemset contains k items, it is called a k-itemset. For instance, {`Beer`, `Diapers`, `Milk`} is an example of a 3-itemset. The null (or empty) set is an itemset that does not contain any items.

A transaction t_j is said to contain an itemset X if X is a subset of t_j. For example, the second transaction shown in Table 4.2 contains the itemset {`Bread`, `Diapers`} but not {`Bread`, `Milk`}. An important property of an itemset is its support count, which refers to the number of transactions that contain a particular itemset. Mathematically, the support count, $\sigma(X)$, for an itemset X can be stated as follows:

$$\sigma(X) = \big|\{t_i | X \subseteq t_i,\ t_i \in T\}\big|,$$

where the symbol $|\cdot|$ denotes the number of elements in a set. In the data set shown in Table 4.2, the support count for {`Beer`, `Diapers`, `Milk`} is equal to two because there are only two transactions that contain all three items.

Often, the property of interest is the support, which is fraction of transactions in which an itemset occurs:

$$s(X) = \sigma(X)/N.$$

An itemset X is called frequent if $s(X)$ is greater than some user-defined threshold, *minsup*.

Association Rule An association rule is an implication expression of the form $X \longrightarrow Y$, where X and Y are disjoint itemsets, i.e., $X \cap Y = \emptyset$. The strength of an association rule can be measured in terms of its **support** and **confidence**. Support determines how often a rule is applicable to a given data set, while confidence determines how frequently items in Y appear in transactions that contain X. The formal definitions of these metrics are

$$\text{Support, } s(X \longrightarrow Y) = \frac{\sigma(X \cup Y)}{N}; \tag{4.1}$$

$$\text{Confidence, } c(X \longrightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)}. \tag{4.2}$$

Example 4.1. Consider the rule {`Milk`, `Diapers`} $\longrightarrow$ {`Beer`}. Because the support count for {`Milk`, `Diapers`, `Beer`} is 2 and the total number of transactions is 5, the rule's support is $2/5 = 0.4$. The rule's confidence is obtained by dividing the support count for {`Milk`, `Diapers`, `Beer`} by the support count for {`Milk`, `Diapers`}. Since there are 3 transactions that contain milk and diapers, the confidence for this rule is $2/3 = 0.67$. ■

Why Use Support and Confidence? Support is an important measure because a rule that has very low support might occur simply by chance. Also, from a business perspective a low support rule is unlikely to be interesting because it might not be profitable to promote items that customers seldom buy together (with the exception of the situation described in Section 4.8). For these reasons, we are interested in finding rules whose support is greater than some user-defined threshold. As will be shown in Section 4.2.1, support also has a desirable property that can be exploited for the efficient discovery of association rules.

Confidence, on the other hand, measures the reliability of the inference made by a rule. For a given rule $X \longrightarrow Y$, the higher the confidence, the more likely it is for Y to be present in transactions that contain X. Confidence also provides an estimate of the conditional probability of Y given X.

Association analysis results should be interpreted with caution. The inference made by an association rule does not necessarily imply causality. Instead, it can sometimes suggest a strong co-occurrence relationship between items in the antecedent and consequent of the rule. Causality, on the other hand, requires knowledge about which attributes in the data capture cause and effect, and typically involves relationships occurring over time (e.g., greenhouse gas emissions lead to global warming). See Section 4.7.1 for additional discussion.

Formulation of the Association Rule Mining Problem The association rule mining problem can be formally stated as follows:

Definition 4.1 (Association Rule Discovery)**.** Given a set of transactions T, find all the rules having support $\geq minsup$ and confidence $\geq minconf$, where $minsup$ and $minconf$ are the corresponding support and confidence thresholds.

A brute-force approach for mining association rules is to compute the support and confidence for every possible rule. This approach is prohibitively expensive because there are exponentially many rules that can be extracted from a data set. More specifically, assuming that neither the left nor the right-hand side of the rule is an empty set, the total number of possible rules, R, extracted from a data set that contains d items is

$$R = 3^d - 2^{d+1} + 1. \tag{4.3}$$

The proof for this equation is left as an exercise to the readers (see Exercise 5 on page 296). Even for the small data set shown in Table 4.1, this approach requires us to compute the support and confidence for $3^6 - 2^7 + 1 = 602$ rules. More than 80% of the rules are discarded after applying $minsup = 20\%$ and $minconf = 50\%$, thus wasting most of the computations. To avoid performing needless computations, it would be useful to prune the rules early without having to compute their support and confidence values.

An initial step toward improving the performance of association rule mining algorithms is to decouple the support and confidence requirements. From Equation 4.1, notice that the support of a rule $X \longrightarrow Y$ is the same as the support of its corresponding itemset, $X \cup Y$. For example, the following rules have identical support because they involve items from the same itemset, {Beer, Diapers, Milk}:

{Beer, Diapers} $\longrightarrow$ {Milk}, {Beer, Milk} $\longrightarrow$ {Diapers},
{Diapers, Milk} $\longrightarrow$ {Beer}, {Beer} $\longrightarrow$ {Diapers, Milk},
{Milk} $\longrightarrow$ {Beer,Diapers}, {Diapers} $\longrightarrow$ {Beer,Milk}.

If the itemset is infrequent, then all six candidate rules can be pruned immediately without our having to compute their confidence values.

Therefore, a common strategy adopted by many association rule mining algorithms is to decompose the problem into two major subtasks:

1. **Frequent Itemset Generation**, whose objective is to find all the itemsets that satisfy the $minsup$ threshold.

2. **Rule Generation**, whose objective is to extract all the high confidence rules from the frequent itemsets found in the previous step. These rules are called strong rules.

The computational requirements for frequent itemset generation are generally more expensive than those of rule generation. Efficient techniques for generating frequent itemsets and association rules are discussed in Sections 4.2 and 4.3, respectively.

4.2 Frequent Itemset Generation

A lattice structure can be used to enumerate the list of all possible itemsets. Figure 4.1 shows an itemset lattice for $I = \{a, b, c, d, e\}$. In general, a data set that contains k items can potentially generate up to $2^k - 1$ frequent itemsets, excluding the null set. Because k can be very large in many practical applications, the search space of itemsets that need to be explored is exponentially large.

A brute-force approach for finding frequent itemsets is to determine the support count for every **candidate itemset** in the lattice structure. To do this, we need to compare each candidate against every transaction, an operation that is shown in Figure 4.2. If the candidate is contained in a transaction, its support count will be incremented. For example, the support for {`Bread`, `Milk`} is incremented three times because the itemset is contained in transactions 1, 4, and 5. Such an approach can be very expensive because it requires $O(NMw)$ comparisons, where N is the number of transactions, $M = 2^k - 1$ is the number of candidate itemsets, and w is the maximum transaction width. **Transaction width** is the number of items present in a transaction.

There are three main approaches for reducing the computational complexity of frequent itemset generation.

1. **Reduce the number of candidate itemsets (M).** The *Apriori* principle, described in the next section, is an effective way to eliminate some of the candidate itemsets without counting their support values.

2. **Reduce the number of comparisons.** Instead of matching each candidate itemset against every transaction, we can reduce the number of comparisons by using more advanced data structures, either to store the candidate itemsets or to compress the data set. We will discuss these strategies in Sections 4.2.4 and 4.6, respectively.

3. **Reduce the number of transactions (N).** As the size of candidate itemsets increases, fewer transactions will be supported by the itemsets.

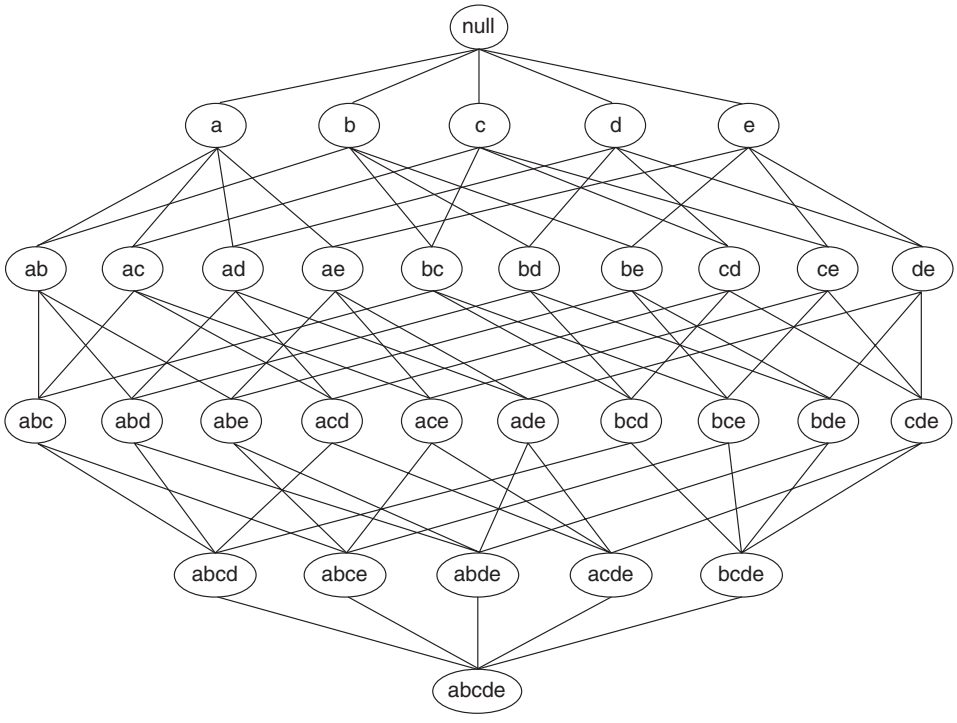


Figure 4.1. An itemset lattice.

For instance, since the width of the first transaction in Table 4.1 is 2, it would be advantageous to remove this transaction before searching for frequent itemsets of size 3 and larger. Algorithms that employ such a strategy are discussed in the Bibliographic Notes.

4.2.1 The *Apriori* Principle

This section describes how the support measure can be used to reduce the number of candidate itemsets explored during frequent itemset generation. The use of support for pruning candidate itemsets is guided by the following principle.

Theorem 4.1 (*Apriori* Principle)**.** *If an itemset is frequent, then all of its subsets must also be frequent.*

To illustrate the idea behind the *Apriori* principle, consider the itemset lattice shown in Figure 4.3. Suppose $\{c, d, e\}$ is a frequent itemset. Clearly, any transaction that contains $\{c, d, e\}$ must also contain its subsets, $\{c, d\}$, $\{c, e\}$,

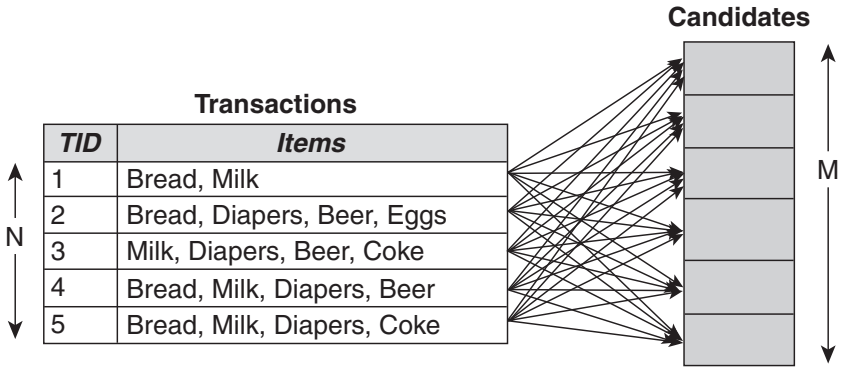


Figure 4.2. Counting the support of candidate itemsets.

$\{d, e\}$, $\{c\}$, $\{d\}$, and $\{e\}$. As a result, if $\{c, d, e\}$ is frequent, then all subsets of $\{c, d, e\}$ (i.e., the shaded itemsets in this figure) must also be frequent.

Conversely, if an itemset such as $\{a, b\}$ is infrequent, then all of its supersets must be infrequent too. As illustrated in Figure 4.4, the entire subgraph containing the supersets of $\{a, b\}$ can be pruned immediately once $\{a, b\}$ is found to be infrequent. This strategy of trimming the exponential search space based on the support measure is known as **support-based pruning**. Such a pruning strategy is made possible by a key property of the support measure, namely, that the support for an itemset never exceeds the support for its subsets. This property is also known as the **anti-monotone** property of the support measure.

Definition 4.2 (Anti-monotone Property)**.** A measure f possesses the anti-monotone property if for every itemset X that is a proper subset of itemset Y, i.e. $X \subset Y$, we have $f(Y) \leq f(X)$.

More generally, a large number of measures–see Section 4.7.1–can be applied to itemsets to evaluate various properties of itemsets. As will be shown in the next section, any measure that has the anti-monotone property can be incorporated directly into an itemset mining algorithm to effectively prune the exponential search space of candidate itemsets.

4.2.2 Frequent Itemset Generation in the *Apriori* Algorithm

Apriori is the first association rule mining algorithm that pioneered the use of support-based pruning to systematically control the exponential growth of candidate itemsets. Figure 4.5 provides a high-level illustration of the frequent itemset generation part of the *Apriori* algorithm for the transactions shown in

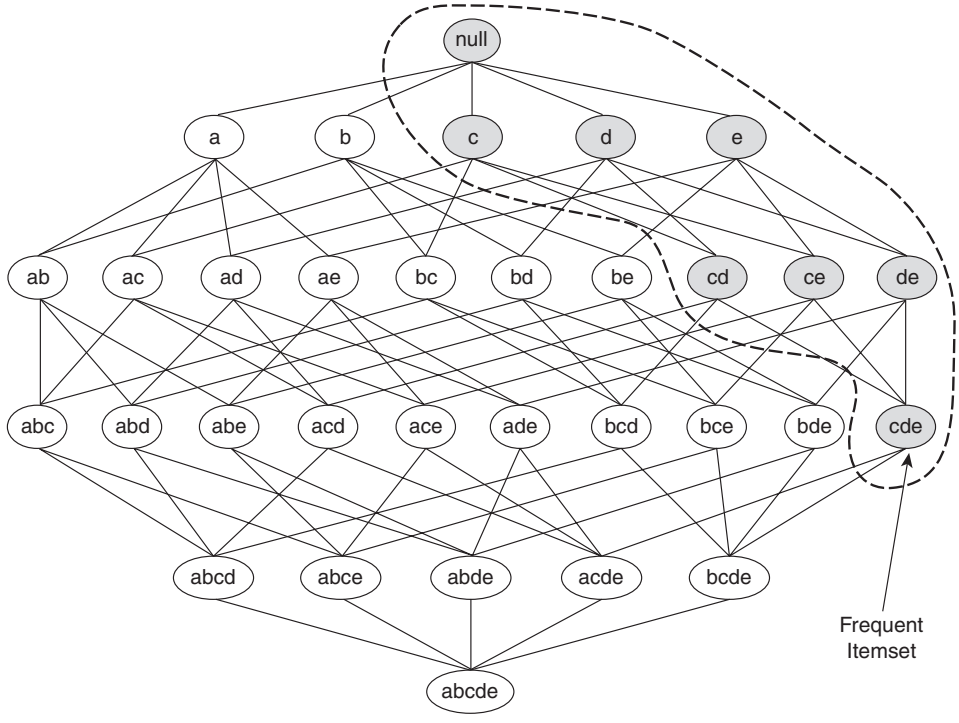


Figure 4.3. An illustration of the *Apriori* principle. If $\{c, d, e\}$ is frequent, then all subsets of this itemset are frequent.

Table 4.1. We assume that the support threshold is 60%, which is equivalent to a minimum support count equal to 3.

Initially, every item is considered as a candidate 1-itemset. After counting their supports, the candidate itemsets {`Cola`} and {`Eggs`} are discarded because they appear in fewer than three transactions. In the next iteration, candidate 2-itemsets are generated using only the frequent 1-itemsets because the *Apriori* principle ensures that all supersets of the infrequent 1-itemsets must be infrequent. Because there are only four frequent 1-itemsets, the number of candidate 2-itemsets generated by the algorithm is $\binom{4}{2} = 6$. Two of these six candidates, {`Beer`, `Bread`} and {`Beer`, `Milk`}, are subsequently found to be infrequent after computing their support values. The remaining four candidates are frequent, and thus will be used to generate candidate 3-itemsets. Without support-based pruning, there are $\binom{6}{3} = 20$ candidate 3-itemsets that can be formed using the six items given in this example. With the *Apriori* principle, we only need to keep candidate 3-itemsets whose subsets are frequent. The only candidate that has this property is {`Bread`,

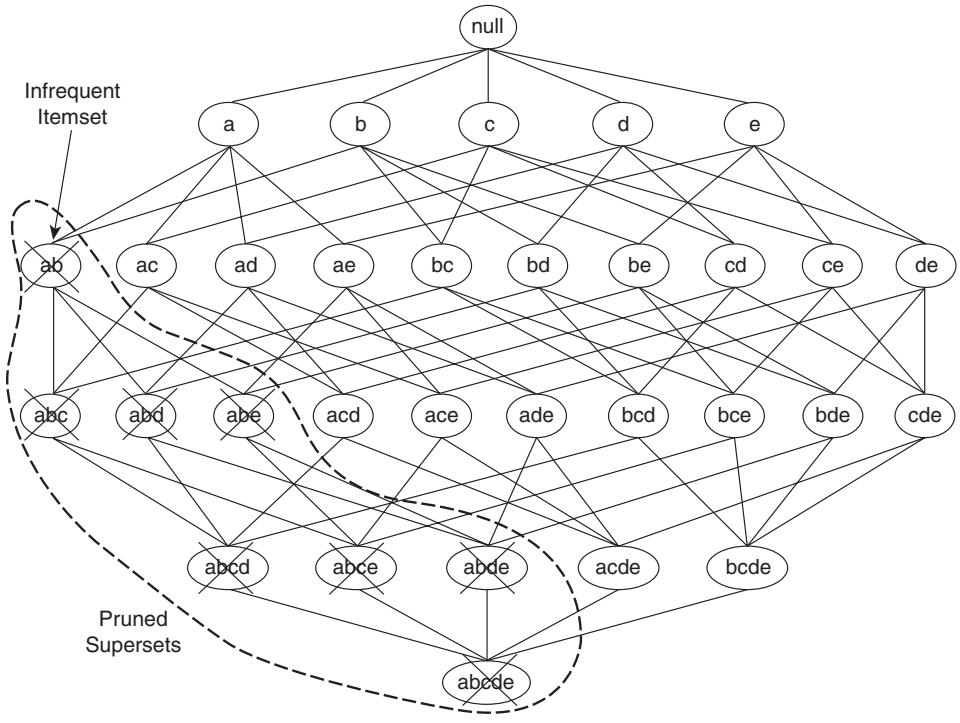


Figure 4.4. An illustration of support-based pruning. If $\{a, b\}$ is infrequent, then all supersets of $\{a, b\}$ are infrequent.

`Diapers`, `Milk`}. However, even though the subsets of {`Bread`, `Diapers`, `Milk`} are frequent, the itemset itself is not.

The effectiveness of the *Apriori* pruning strategy can be shown by counting the number of candidate itemsets generated. A brute-force strategy of enumerating all itemsets (up to size 3) as candidates will produce

$$\binom{6}{1} + \binom{6}{2} + \binom{6}{3} = 6 + 15 + 20 = 41$$

candidates. With the *Apriori* principle, this number decreases to

$$\binom{6}{1} + \binom{4}{2} + 1 = 6 + 6 + 1 = 13$$

candidates, which represents a 68% reduction in the number of candidate itemsets even in this simple example.

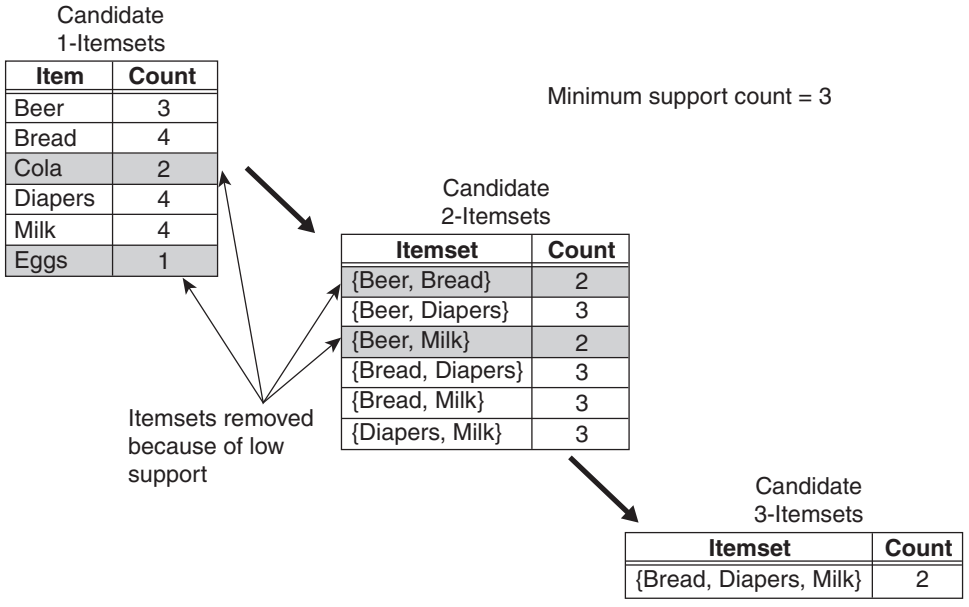


Figure 4.5. Illustration of frequent itemset generation using the *Apriori* algorithm.

The pseudocode for the frequent itemset generation part of the *Apriori* algorithm is shown in Algorithm 4.1. Let C_k denote the set of candidate k-itemsets and F_k denote the set of frequent k-itemsets:

- The algorithm initially makes a single pass over the data set to determine the support of each item. Upon completion of this step, the set of all frequent 1-itemsets, F_1, will be known (steps 1 and 2).

- Next, the algorithm will iteratively generate new candidate k-itemsets and prune unnecessary candidates that are guaranteed to be infrequent given the frequent $(k-1)$-itemsets found in the previous iteration (steps 5 and 6). Candidate generation and pruning is implemented using the functions candidate-gen and candidate-prune, which are described in Section 4.2.3.

- To count the support of the candidates, the algorithm needs to make an additional pass over the data set (steps 7–12). The subset function is used to determine all the candidate itemsets in C_k that are contained in each transaction t. The implementation of this function is described in Section 4.2.4.

- After counting their supports, the algorithm eliminates all candidate itemsets whose support counts are less than $N \times minsup$ (step 13).
- The algorithm terminates when there are no new frequent itemsets generated, i.e., $F_k = \emptyset$ (step 14).

The frequent itemset generation part of the *Apriori* algorithm has two important characteristics. First, it is a **level-wise** algorithm; i.e., it traverses the itemset lattice one level at a time, from frequent 1-itemsets to the maximum size of frequent itemsets. Second, it employs a **generate-and-test** strategy for finding frequent itemsets. At each iteration (level), new candidate itemsets are generated from the frequent itemsets found in the previous iteration. The support for each candidate is then counted and tested against the *minsup* threshold. The total number of iterations needed by the algorithm is $k_{\max} + 1$, where $k_{\max}$ is the maximum size of the frequent itemsets.

4.2.3 Candidate Generation and Pruning

The candidate-gen and candidate-prune functions shown in Steps 5 and 6 of Algorithm 4.1 generate candidate itemsets and prunes unnecessary ones by performing the following two operations, respectively:

1. **Candidate Generation.** This operation generates new candidate k-itemsets based on the frequent $(k-1)$-itemsets found in the previous iteration.

Algorithm 4.1 Frequent itemset generation of the *Apriori* algorithm.

1: $k = 1$.
2: $F_k = \{ i \mid i \in I \wedge \sigma(\{i\}) \geq N \times minsup\}$. {Find all frequent 1-itemsets}
3: **repeat**
4: $k = k + 1$.
5: C_k = candidate-gen(F_{k-1}). {Generate candidate itemsets.}
6: C_k = candidate-prune(C_k, F_{k-1}). {Prune candidate itemsets.}
7: **for** each transaction $t \in T$ **do**
8: C_t = subset(C_k, t). {Identify all candidates that belong to t.}
9: **for** each candidate itemset $c \in C_t$ **do**
10: $\sigma(c) = \sigma(c) + 1$. {Increment support count.}
11: **end for**
12: **end for**
13: $F_k = \{ c \mid c \in C_k \wedge \sigma(c) \geq N \times minsup\}$. {Extract the frequent k-itemsets.}
14: **until** $F_k = \emptyset$
15: Result = $\bigcup F_k$.

2. **Candidate Pruning.** This operation eliminates some of the candidate k-itemsets using support-based pruning, i.e. by removing k-itemsets whose subsets are known to be infrequent in previous iterations. Note that this pruning is done without computing the actual support of these k-itemsets (which could have required comparing them against each transaction).

Candidate Generation

In principle, there are many ways to generate candidate itemsets. An effective candidate generation procedure must be complete and non-redundant. A candidate generation procedure is said to be *complete* if it does not omit any frequent itemsets. To ensure completeness, the set of candidate itemsets must subsume the set of all frequent itemsets, i.e., $\forall k : F_k \subseteq C_k$. A candidate generation procedure is *non-redundant* if it does not generate the same candidate itemset more than once. For example, the candidate itemset $\{a, b, c, d\}$ can be generated in many ways—by merging $\{a, b, c\}$ with $\{d\}$, $\{b, d\}$ with $\{a, c\}$, $\{c\}$ with $\{a, b, d\}$, etc. Generation of duplicate candidates leads to wasted computations and thus should be avoided for efficiency reasons. Also, an effective candidate generation procedure should avoid generating too many unnecessary candidates. A candidate itemset is unnecessary if at least one of its subsets is infrequent, and thus, eliminated in the candidate pruning step.

Next, we will briefly describe several candidate generation procedures, including the one used by the candidate-gen function.

Brute-Force Method The brute-force method considers every k-itemset as a potential candidate and then applies the candidate pruning step to remove any unnecessary candidates whose subsets are infrequent (see Figure 4.6). The number of candidate itemsets generated at level k is equal to $\binom{d}{k}$, where d is the total number of items. Although candidate generation is rather trivial, candidate pruning becomes extremely expensive because a large number of itemsets must be examined.

$\mathbf{F}_{k-1} \times \mathbf{F}_1$ Method An alternative method for candidate generation is to extend each frequent $(k-1)$-itemset with frequent items that are not part of the $(k-1)$-itemset. Figure 4.7 illustrates how a frequent 2-itemset such as {`Beer`, `Diapers`} can be augmented with a frequent item such as `Bread` to produce a candidate 3-itemset {`Beer`, `Diapers`, `Bread`}.

The procedure is complete because every frequent k-itemset is composed of a frequent $(k-1)$-itemset and a frequent 1-itemset. Therefore, all frequent

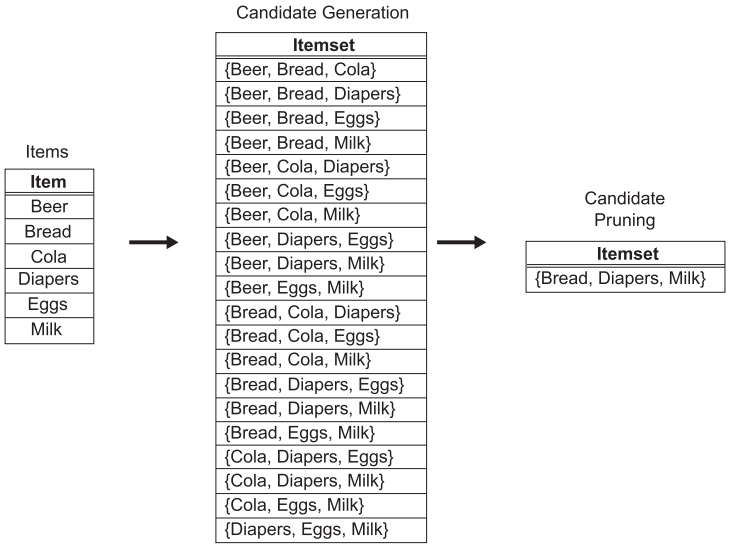


Figure 4.6. A brute-force method for generating candidate 3-itemsets.

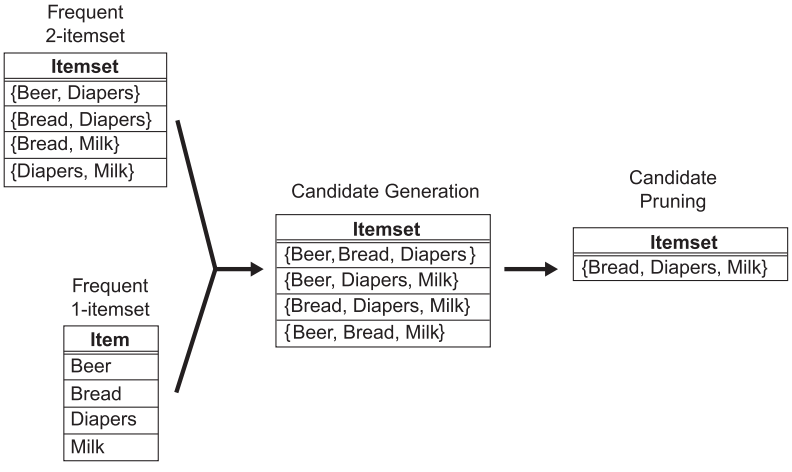


Figure 4.7. Generating and pruning candidate k-itemsets by merging a frequent $(k-1)$-itemset with a frequent item. Note that some of the candidates are unnecessary because their subsets are infrequent.

k-itemsets are part of the candidate k-itemsets generated by this procedure. Figure 4.7 shows that the $\mathbf{F}_{k-1} \times \mathbf{F}_1$ candidate generation method only produces four candidate 3-itemsets, instead of the $\binom{6}{3} = 20$ itemsets produced by the brute-force method. The $\mathbf{F}_{k-1} \times \mathbf{F}_1$ method generates lower number

of candidates because every candidate is guaranteed to contain at least one frequent $(k-1)$-itemset. While this procedure is a substantial improvement over the brute-force method, it can still produce a large number of unnecessary candidates, as the remaining subsets of a candidate itemset can still be infrequent.

Note that the approach discussed above does not prevent the same candidate itemset from being generated more than once. For instance, {`Bread`, `Diapers`, `Milk`} can be generated by merging {`Bread`, `Diapers`} with {`Milk`}, {`Bread`, `Milk`} with {`Diapers`}, or {`Diapers`, `Milk`} with {`Bread`}. One way to avoid generating duplicate candidates is by ensuring that the items in each frequent itemset are kept sorted in their lexicographic order. For example, itemsets such as {`Bread`, `Diapers`}, {`Bread`, `Diapers`, `Milk`}, and {`Diapers`, `Milk`} follow lexicographic order as the items within every itemset are arranged alphabetically. Each frequent $(k-1)$-itemset X is then extended with frequent items that are lexicographically larger than the items in X. For example, the itemset {`Bread`, `Diapers`} can be augmented with {Milk} because `Milk` is lexicographically larger than `Bread` and `Diapers`. However, we should not augment {`Diapers`, `Milk`} with {`Bread`} nor {`Bread`, `Milk`} with {`Diapers`} because they violate the lexicographic ordering condition. Every candidate k-itemset is thus generated exactly once, by merging the lexicographically largest item with the remaining $k-1$ items in the itemset. If the $\mathbf{F}_{k-1} \times \mathbf{F}_1$ method is used in conjunction with lexicographic ordering, then only two candidate 3-itemsets will be produced in the example illustrated in Figure 4.7. {`Beer`, `Bread`, `Diapers`} and {`Beer`, `Bread`, `Milk`} will not be generated because {`Beer`, `Bread`} is not a frequent 2-itemset.

$\mathbf{F}_{k-1} \times \mathbf{F}_{k-1}$ Method This candidate generation procedure, which is used in the candidate-gen function of the *Apriori* algorithm, merges a pair of frequent $(k-1)$-itemsets only if their first $k-2$ items, arranged in lexicographic order, are identical. Let $A = \{a_1, a_2, \ldots, a_{k-1}\}$ and $B = \{b_1, b_2, \ldots, b_{k-1}\}$ be a pair of frequent $(k-1)$-itemsets, arranged lexicographically. A and B are merged if they satisfy the following conditions:

$$a_i = b_i \text{ (for } i = 1, 2, \ldots, k-2).$$

Note that in this case, $a_{k-1} \neq b_{k-1}$ because A and B are two distinct itemsets. The candidate k-itemset generated by merging A and B consists of the first $k-2$ common items followed by a_{k-1} and b_{k-1} in lexicographic order. This candidate generation procedure is complete, because for every lexicographically ordered frequent k-itemset, there exists two lexicographically

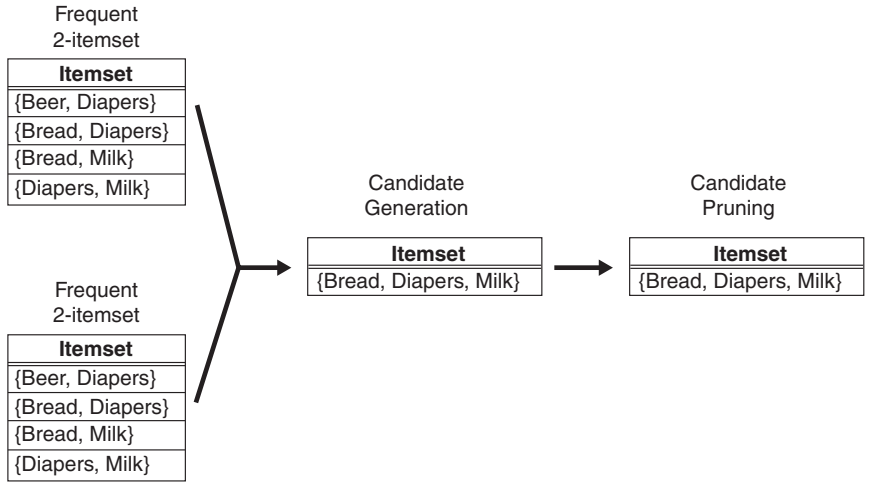


Figure 4.8. Generating and pruning candidate k-itemsets by merging pairs of frequent $(k-1)$-itemsets.

ordered frequent $(k-1)$-itemsets that have identical items in the first $k-2$ positions.

In Figure 4.8, the frequent itemsets {`Bread`, `Diapers`} and {`Bread`, `Milk`} are merged to form a candidate 3-itemset {`Bread`, `Diapers`, `Milk`}. The algorithm does not have to merge {`Beer`, `Diapers`} with {`Diapers`, `Milk`} because the first item in both itemsets is different. Indeed, if {`Beer`, `Diapers`, `Milk`} is a viable candidate, it would have been obtained by merging {`Beer`, `Diapers`} with {`Beer`, `Milk`} instead. This example illustrates both the completeness of the candidate generation procedure and the advantages of using lexicographic ordering to prevent duplicate candidates. Also, if we order the frequent $(k-1)$-itemsets according to their lexicographic rank, itemsets with identical first $k-2$ items would take consecutive ranks. As a result, the $\mathbf{F}_{k-1} \times \mathbf{F}_{k-1}$ candidate generation method would consider merging a frequent itemset only with ones that occupy the next few ranks in the sorted list, thus saving some computations.

Figure 4.8 shows that the $\mathbf{F}_{k-1} \times \mathbf{F}_{k-1}$ candidate generation procedure results in only one candidate 3-itemset. This is a considerable reduction from the four candidate 3-itemsets generated by the $\mathbf{F}_{k-1} \times \mathbf{F}_1$ method. This is because the $\mathbf{F}_{k-1} \times \mathbf{F}_{k-1}$ method ensures that every candidate k-itemset contains at least two frequent $(k-1)$-itemsets, thus greatly reducing the number of candidates that are generated in this step.

Note that there can be multiple ways of merging two frequent $(k-1)$-itemsets in the $\mathbf{F}_{k-1} \times \mathbf{F}_{k-1}$ procedure, one of which is merging if their first

$k-2$ items are identical. An alternate approach could be to merge two frequent $(k-1)$-itemsets A and B if the last $k-2$ items of A are identical to the first $k-2$ itemsets of B. For example, {`Bread`, `Diapers`} and {`Diapers`, `Milk`} could be merged using this approach to generate the candidate 3-itemset {`Bread`, `Diapers`, `Milk`}. As we will see later, this alternate $\mathbf{F}_{k-1} \times \mathbf{F}_{k-1}$ procedure is useful in generating sequential patterns, which will be discussed in Chapter 7.

Candidate Pruning

To illustrate the candidate pruning operation for a candidate k-itemset, $X = \{i_1, i_2, \ldots, i_k\}$, consider its k proper subsets, $X - \{i_j\}$ $(\forall j = 1, 2, \ldots, k)$. If any of them are infrequent, then X is immediately pruned by using the *Apriori* principle. Note that we don't need to explicitly ensure that all subsets of X of size less than $k-1$ are frequent (see Exercise 8). This approach greatly reduces the number of candidate itemsets considered during support counting. For the brute-force candidate generation method, candidate pruning requires checking only k subsets of size $k-1$ for each candidate k-itemset. However, since the $\mathbf{F}_{k-1} \times \mathbf{F}_1$ candidate generation strategy ensures that at least one of the $(k-1)$-size subsets of every candidate k-itemset is frequent, we only need to check for the remaining $k-1$ subsets. Likewise, the $\mathbf{F}_{k-1} \times \mathbf{F}_{k-1}$ strategy requires examining only $k-2$ subsets of every candidate k-itemset, since two of its $(k-1)$-size subsets are already known to be frequent in the candidate generation step.

4.2.4 Support Counting

Support counting is the process of determining the frequency of occurrence for every candidate itemset that survives the candidate pruning step. Support counting is implemented in steps 6 through 11 of Algorithm 4.1. A brute-force approach for doing this is to compare each transaction against every candidate itemset (see Figure 4.2) and to update the support counts of candidates contained in a transaction. This approach is computationally expensive, especially when the numbers of transactions and candidate itemsets are large.

An alternative approach is to enumerate the itemsets contained in each transaction and use them to update the support counts of their respective candidate itemsets. To illustrate, consider a transaction t that contains five items, $\{1, 2, 3, 5, 6\}$. There are $\binom{5}{3} = 10$ itemsets of size 3 contained in this transaction. Some of the itemsets may correspond to the candidate 3-itemsets under investigation, in which case, their support counts are incremented. Other subsets of t that do not correspond to any candidates can be ignored.

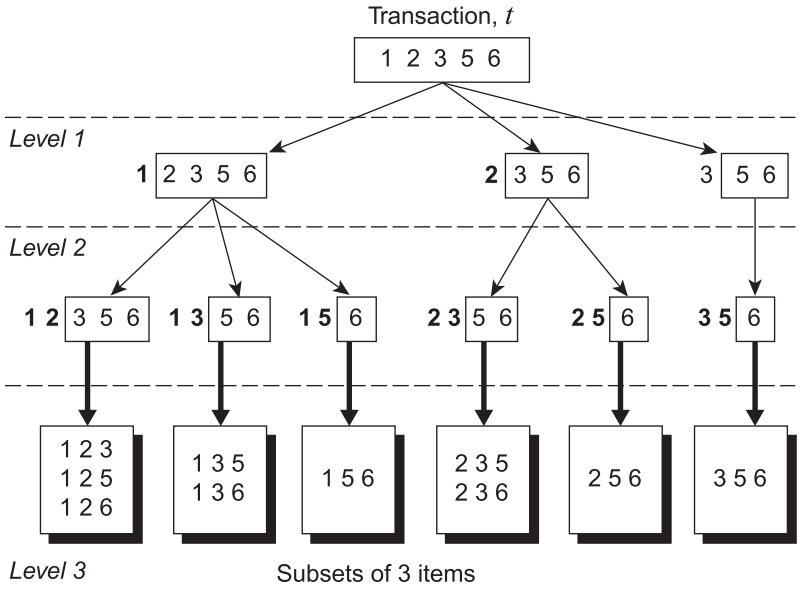


Figure 4.9. Enumerating subsets of three items from a transaction t.

Figure 4.9 shows a systematic way for enumerating the 3-itemsets contained in t. Assuming that each itemset keeps its items in increasing lexicographic order, an itemset can be enumerated by specifying the smallest item first, followed by the larger items. For instance, given $t = \{1, 2, 3, 5, 6\}$, all the 3-itemsets contained in t must begin with item 1, 2, or 3. It is not possible to construct a 3-itemset that begins with items 5 or 6 because there are only two items in t whose labels are greater than or equal to 5. The number of ways to specify the first item of a 3-itemset contained in t is illustrated by the Level 1 prefix tree structure depicted in Figure 4.9. For instance, 1 $\boxed{2\ 3\ 5\ 6}$ represents a 3-itemset that begins with item 1, followed by two more items chosen from the set $\{2, 3, 5, 6\}$.

After fixing the first item, the prefix tree structure at Level 2 represents the number of ways to select the second item. For example, 1 2 $\boxed{3\ 5\ 6}$ corresponds to itemsets that begin with the prefix (1 2) and are followed by the items 3, 5, or 6. Finally, the prefix tree structure at Level 3 represents the complete set of 3-itemsets contained in t. For example, the 3-itemsets that begin with prefix $\{1\ 2\}$ are $\{1, 2, 3\}$, $\{1, 2, 5\}$, and $\{1, 2, 6\}$, while those that begin with prefix $\{2\ 3\}$ are $\{2, 3, 5\}$ and $\{2, 3, 6\}$.

The prefix tree structure shown in Figure 4.9 demonstrates how itemsets contained in a transaction can be systematically enumerated, i.e., by specifying

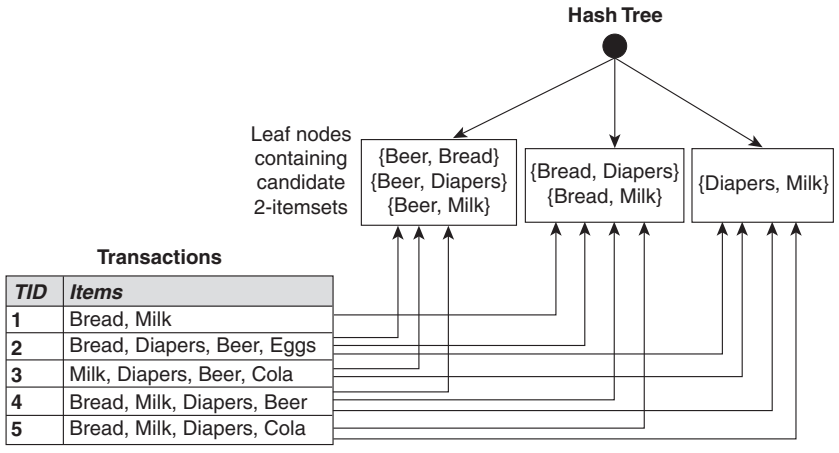


Figure 4.10. Counting the support of itemsets using hash structure.

their items one by one, from the leftmost item to the rightmost item. We still have to determine whether each enumerated 3-itemset corresponds to an existing candidate itemset. If it matches one of the candidates, then the support count of the corresponding candidate is incremented. In the next section, we illustrate how this matching operation can be performed efficiently using a hash tree structure.

Support Counting Using a Hash Tree*

In the *Apriori* algorithm, candidate itemsets are partitioned into different buckets and stored in a hash tree. During support counting, itemsets contained in each transaction are also hashed into their appropriate buckets. That way, instead of comparing each itemset in the transaction with every candidate itemset, it is matched only against candidate itemsets that belong to the same bucket, as shown in Figure 4.10.

Figure 4.11 shows an example of a hash tree structure. Each internal node of the tree uses the following hash function, $h(p) = (p - 1) \bmod 3$, where mode refers to the modulo (remainder) operator, to determine which branch of the current node should be followed next. For example, items 1, 4, and 7 are hashed to the same branch (i.e., the leftmost branch) because they have the same remainder after dividing the number by 3. All candidate itemsets are stored at the leaf nodes of the hash tree. The hash tree shown in Figure 4.11 contains 15 candidate 3-itemsets, distributed across 9 leaf nodes.

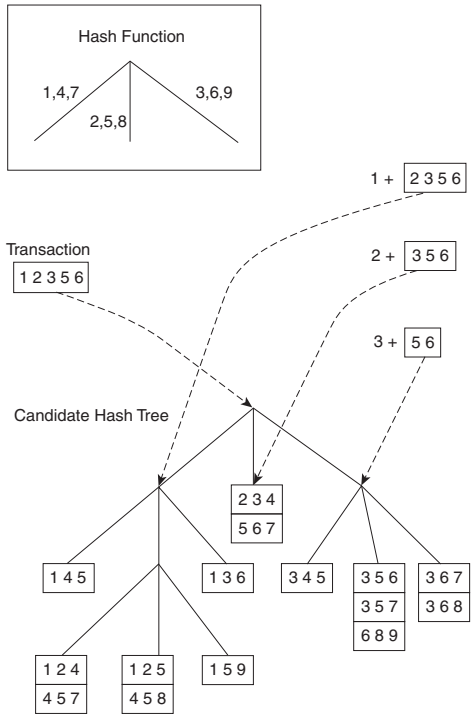


Figure 4.11. Hashing a transaction at the root node of a hash tree.

Consider the transaction, $t = \{1, 2, 3, 5, 6\}$. To update the support counts of the candidate itemsets, the hash tree must be traversed in such a way that all the leaf nodes containing candidate 3-itemsets belonging to t must be visited at least once. Recall that the 3-itemsets contained in t must begin with items 1, 2, or 3, as indicated by the Level 1 prefix tree structure shown in Figure 4.9. Therefore, at the root node of the hash tree, the items 1, 2, and 3 of the transaction are hashed separately. Item 1 is hashed to the left child of the root node, item 2 is hashed to the middle child, and item 3 is hashed to the right child. At the next level of the tree, the transaction is hashed on the second item listed in the Level 2 tree structure shown in Figure 4.9. For example, after hashing on item 1 at the root node, items 2, 3, and 5 of the transaction are hashed. Based on the hash function, items 2 and 5 are hashed to the middle child, while item 3 is hashed to the right child, as shown in Figure 4.12. This process continues until the leaf nodes of the hash tree are reached. The candidate itemsets stored at the visited leaf nodes are compared against the transaction. If a candidate is a subset of the transaction, its support count is incremented. Note that not all the leaf nodes are visited

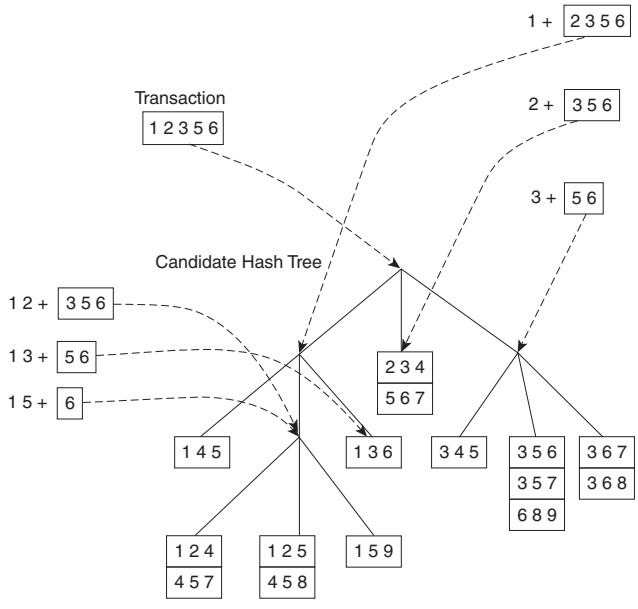


Figure 4.12. Subset operation on the leftmost subtree of the root of a candidate hash tree.

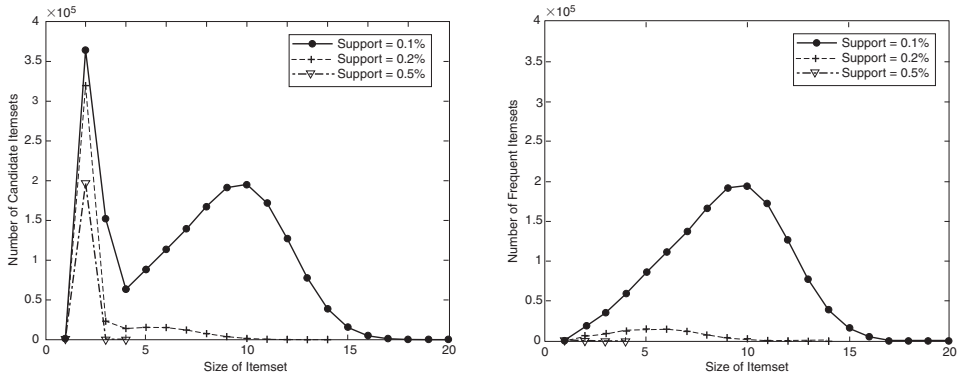
while traversing the hash tree, which helps in reducing the computational cost. In this example, 5 out of the 9 leaf nodes are visited and 9 out of the 15 itemsets are compared against the transaction.

4.2.5 Computational Complexity

The computational complexity of the *Apriori* algorithm, which includes both its runtime and storage, can be affected by the following factors.

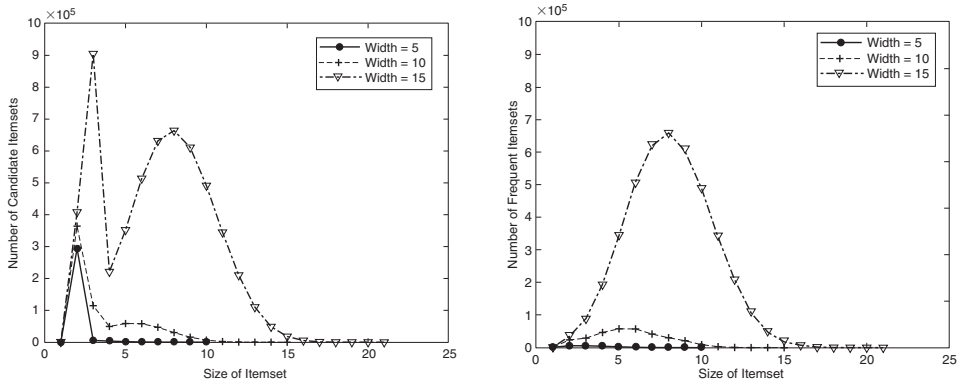
Support Threshold Lowering the support threshold often results in more itemsets being declared as frequent. This has an adverse effect on the computational complexity of the algorithm because more candidate itemsets must be generated and counted at every level, as shown in Figure 4.13. The maximum size of frequent itemsets also tends to increase with lower support thresholds. This increases the total number of iterations to be performed by the *Apriori* algorithm, further increasing the computational cost.

Number of Items (Dimensionality) As the number of items increases, more space will be needed to store the support counts of items. If the number of frequent items also grows with the dimensionality of the data, the runtime and



(a) Number of candidate itemsets. (b) Number of frequent itemsets.

Figure 4.13. Effect of support threshold on the number of candidate and frequent itemsets obtained from a benchmark data set.



(a) Number of candidate itemsets. (b) Number of Frequent Itemsets.

Figure 4.14. Effect of average transaction width on the number of candidate and frequent itemsets obtained from a synthetic data set.

storage requirements will increase because of the larger number of candidate itemsets generated by the algorithm.

Number of Transactions Because the *Apriori* algorithm makes repeated passes over the transaction data set, its run time increases with a larger number of transactions.

Average Transaction Width For dense data sets, the average transaction width can be very large. This affects the complexity of the *Apriori* algorithm in two ways. First, the maximum size of frequent itemsets tends to increase as the average transaction width increases. As a result, more candidate itemsets must be examined during candidate generation and support counting, as illustrated in Figure 4.14. Second, as the transaction width increases, more itemsets are contained in the transaction. This will increase the number of hash tree traversals performed during support counting.

A detailed analysis of the time complexity for the *Apriori* algorithm is presented next.

Generation of frequent 1-itemsets For each transaction, we need to update the support count for every item present in the transaction. Assuming that w is the average transaction width, this operation requires $O(Nw)$ time, where N is the total number of transactions.

Candidate generation To generate candidate k-itemsets, pairs of frequent $(k-1)$-itemsets are merged to determine whether they have at least $k-2$ items in common. Each merging operation requires at most $k-2$ equality comparisons. Every merging step can produce at most one viable candidate k-itemset, while in the worst-case, the algorithm must try to merge every pair of frequent $(k-1)$-itemsets found in the previous iteration. Therefore, the overall cost of merging frequent itemsets is

$$\sum_{k=2}^{w}(k-2)|C_k| < \text{ Cost of merging } < \sum_{k=2}^{w}(k-2)|F_{k-1}|^2,$$

where w is the maximum transaction width. A hash tree is also constructed during candidate generation to store the candidate itemsets. Because the maximum depth of the tree is k, the cost for populating the hash tree with candidate itemsets is $O\big(\sum_{k=2}^{w} k|C_k|\big)$. During candidate pruning, we need to verify that the $k-2$ subsets of every candidate k-itemset are frequent. Since the cost for looking up a candidate in a hash tree is $O(k)$, the candidate pruning step requires $O\big(\sum_{k=2}^{w} k(k-2)|C_k|\big)$ time.

Support counting Each transaction of width $|t|$ produces $\binom{|t|}{k}$ itemsets of size k. This is also the effective number of hash tree traversals performed for each transaction. The cost for support counting is $O\left(N \sum_k \binom{w}{k} \alpha_k\right)$, where w is the maximum transaction width and α_k is the cost for updating the support count of a candidate k-itemset in the hash tree.

4.3 Rule Generation

This section describes how to extract association rules efficiently from a given frequent itemset. Each frequent k-itemset, Y, can produce up to $2^k - 2$ association rules, ignoring rules that have empty antecedents or consequents ($\emptyset \longrightarrow Y$ or $Y \longrightarrow \emptyset$). An association rule can be extracted by partitioning the itemset Y into two non-empty subsets, X and $Y - X$, such that $X \longrightarrow Y - X$ satisfies the confidence threshold. Note that all such rules must have already met the support threshold because they are generated from a frequent itemset.

Example 4.2. Let $X = \{a, b, c\}$ be a frequent itemset. There are six candidate association rules that can be generated from X: $\{a, b\} \longrightarrow \{c\}$, $\{a, c\} \longrightarrow \{b\}$, $\{b, c\} \longrightarrow \{a\}$, $\{a\} \longrightarrow \{b, c\}$, $\{b\} \longrightarrow \{a, c\}$, and $\{c\} \longrightarrow \{a, b\}$. As each of their support is identical to the support for X, all the rules satisfy the support threshold. ■

Computing the confidence of an association rule does not require additional scans of the transaction data set. Consider the rule $\{1, 2\} \longrightarrow \{3\}$, which is generated from the frequent itemset $X = \{1, 2, 3\}$. The confidence for this rule is $\sigma(\{1, 2, 3\})/\sigma(\{1, 2\})$. Because $\{1, 2, 3\}$ is frequent, the anti-monotone property of support ensures that $\{1, 2\}$ must be frequent, too. Since the support counts for both itemsets were already found during frequent itemset generation, there is no need to read the entire data set again.

4.3.1 Confidence-Based Pruning

Confidence does not show the anti-monotone property in the same way as the support measure. For example, the confidence for $X \longrightarrow Y$ can be larger, smaller, or equal to the confidence for another rule $\tilde{X} \longrightarrow \tilde{Y}$, where $\tilde{X} \subseteq X$ and $\tilde{Y} \subseteq Y$ (see Exercise 3 on page 295). Nevertheless, if we compare rules generated from the same frequent itemset Y, the following theorem holds for the confidence measure.

Theorem 4.2. *Let Y be an itemset and X is a subset of Y. If a rule $X \longrightarrow Y - X$ does not satisfy the confidence threshold, then any rule $\tilde{X} \longrightarrow Y - \tilde{X}$, where $\tilde{X}$ is a subset of X, must not satisfy the confidence threshold as well.*

To prove this theorem, consider the following two rules: $\tilde{X} \longrightarrow Y - \tilde{X}$ and $X \longrightarrow Y - X$, where $\tilde{X} \subset X$. The confidence of the rules are $\sigma(Y)/\sigma(\tilde{X})$ and $\sigma(Y)/\sigma(X)$, respectively. Since $\tilde{X}$ is a subset of X, $\sigma(\tilde{X}) \geq \sigma(X)$. Therefore, the former rule cannot have a higher confidence than the latter rule.

4.3.2 Rule Generation in *Apriori* Algorithm

The *Apriori* algorithm uses a level-wise approach for generating association rules, where each level corresponds to the number of items that belong to the rule consequent. Initially, all the high confidence rules that have only one item in the rule consequent are extracted. These rules are then used to generate new candidate rules. For example, if $\{acd\} \longrightarrow \{b\}$ and $\{abd\} \longrightarrow \{c\}$ are high confidence rules, then the candidate rule $\{ad\} \longrightarrow \{bc\}$ is generated by merging the consequents of both rules. Figure 4.15 shows a lattice structure for the association rules generated from the frequent itemset $\{a, b, c, d\}$. If any node in the lattice has low confidence, then according to Theorem 4.2, the entire subgraph spanned by the node can be pruned immediately. Suppose the confidence for $\{bcd\} \longrightarrow \{a\}$ is low. All the rules containing item a in its consequent, including $\{cd\} \longrightarrow \{ab\}$, $\{bd\} \longrightarrow \{ac\}$, $\{bc\} \longrightarrow \{ad\}$, and $\{d\} \longrightarrow \{abc\}$ can be discarded.

A pseudocode for the rule generation step is shown in Algorithms 4.2 and 4.3. Note the similarity between the `ap-genrules` procedure given in Algorithm 4.3 and the frequent itemset generation procedure given in Algorithm 4.1. The only difference is that, in rule generation, we do not have to make additional passes over the data set to compute the confidence of the candidate rules. Instead, we determine the confidence of each rule by using the support counts computed during frequent itemset generation.

Algorithm 4.2 Rule generation of the *Apriori* algorithm.

1: **for** each frequent k-itemset f_k, $k \geq 2$ **do**
2: $H_1 = \{i \mid i \in f_k\}$ {1-item consequents of the rule.}
3: **call** `ap-genrules`$(f_k, H_1.)$
4: **end for**

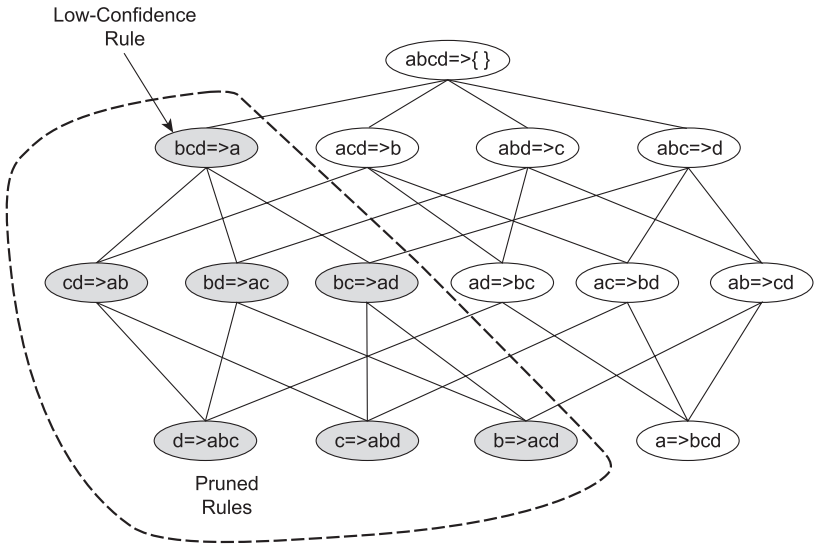


Figure 4.15. Pruning of association rules using the confidence measure.

Algorithm 4.3 Procedure `ap-genrules`(f_k, H_m).

1: $k = |f_k|$ {size of frequent itemset.}
2: $m = |H_m|$ {size of rule consequent.}
3: **if** $k > m + 1$ **then**
4: H_{m+1} = candidate-gen(H_m).
5: H_{m+1} = candidate-prune(H_{m+1},H_m).
6: **for** each $h_{m+1} \in H_{m+1}$ **do**
7: $conf = \sigma(f_k)/\sigma(f_k - h_{m+1})$.
8: **if** $conf \geq minconf$ **then**
9: **output** the rule $(f_k - h_{m+1}) \longrightarrow h_{m+1}$.
10: **else**
11: **delete** h_{m+1} from H_{m+1}.
12: **end if**
13: **end for**
14: **call** ap-genrules(f_k, H_{m+1}.)
15: **end if**

4.3.3 An Example: Congressional Voting Records

This section demonstrates the results of applying association analysis to the voting records of members of the United States House of Representatives. The data is obtained from the 1984 Congressional Voting Records Database, which is available at the UCI machine learning data repository. Each transaction

contains information about the party affiliation for a representative along with his or her voting record on 16 key issues. There are 435 transactions and 34 items in the data set. The set of items are listed in Table 4.3.

Table 4.3. List of binary attributes from the 1984 United States Congressional Voting Records. Source: The UCI machine learning repository.

1. Republican	18. aid to Nicaragua = no
2. Democrat	19. MX-missile = yes
3. handicapped-infants = yes	20. MX-missile = no
4. handicapped-infants = no	21. immigration = yes
5. water project cost sharing = yes	22. immigration = no
6. water project cost sharing = no	23. synfuel corporation cutback = yes
7. budget-resolution = yes	24. synfuel corporation cutback = no
8. budget-resolution = no	25. education spending = yes
9. physician fee freeze = yes	26. education spending = no
10. physician fee freeze = no	27. right-to-sue = yes
11. aid to El Salvador = yes	28. right-to-sue = no
12. aid to El Salvador = no	29. crime = yes
13. religious groups in schools = yes	30. crime = no
14. religious groups in schools = no	31. duty-free-exports = yes
15. anti-satellite test ban = yes	32. duty-free-exports = no
16. anti-satellite test ban = no	33. export administration act = yes
17. aid to Nicaragua = yes	34. export administration act = no

Table 4.4. Association rules extracted from the 1984 United States Congressional Voting Records.

Association Rule	Confidence
{budget resolution = no, MX-missile=no, aid to El Salvador = yes } $\longrightarrow$ {Republican}	91.0%
{budget resolution = yes, MX-missile=yes, aid to El Salvador = no } $\longrightarrow$ {Democrat}	97.5%
{crime = yes, right-to-sue = yes, physician fee freeze = yes} $\longrightarrow$ {Republican}	93.5%
{crime = no, right-to-sue = no, physician fee freeze = no} $\longrightarrow$ {Democrat}	100%

The *Apriori* algorithm is then applied to the data set with $minsup = 30\%$ and $minconf = 90\%$. Some of the high confidence rules extracted by the

algorithm are shown in Table 4.4. The first two rules suggest that most of the members who voted yes for aid to El Salvador and no for budget resolution and MX missile are Republicans; while those who voted no for aid to El Salvador and yes for budget resolution and MX missile are Democrats. These high confidence rules show the key issues that divide members from both political parties.

4.4 Compact Representation of Frequent Itemsets

In practice, the number of frequent itemsets produced from a transaction data set can be very large. It is useful to identify a small representative set of frequent itemsets from which all other frequent itemsets can be derived. Two such representations are presented in this section in the form of maximal and closed frequent itemsets.

4.4.1 Maximal Frequent Itemsets

Definition 4.3 (Maximal Frequent Itemset)**.** A frequent itemset is maximal if none of its immediate supersets are frequent.

To illustrate this concept, consider the itemset lattice shown in Figure 4.16. The itemsets in the lattice are divided into two groups: those that are frequent and those that are infrequent. A frequent itemset border, which is represented by a dashed line, is also illustrated in the diagram. Every itemset located above the border is frequent, while those located below the border (the shaded nodes) are infrequent. Among the itemsets residing near the border, $\{a, d\}$, $\{a, c, e\}$, and $\{b, c, d, e\}$ are maximal frequent itemsets because all of their immediate supersets are infrequent. For example, the itemset $\{a, d\}$ is maximal frequent because all of its immediate supersets, $\{a, b, d\}$, $\{a, c, d\}$, and $\{a, d, e\}$, are infrequent. In contrast, $\{a, c\}$ is non-maximal because one of its immediate supersets, $\{a, c, e\}$, is frequent.

Maximal frequent itemsets effectively provide a compact representation of frequent itemsets. In other words, they form the smallest set of itemsets from which all frequent itemsets can be derived. For example, every frequent itemset in Figure 4.16 is a subset of one of the three maximal frequent itemsets, $\{a, d\}$, $\{a, c, e\}$, and $\{b, c, d, e\}$. If an itemset is not a proper subset of any of the maximal frequent itemsets, then it is either infrequent (e.g., $\{a, d, e\}$) or maximal frequent itself (e.g., $\{b, c, d, e\}$). Hence, the maximal frequent itemsets $\{a, c, e\}$, $\{a, d\}$, and $\{b, c, d, e\}$ provide a compact representation of

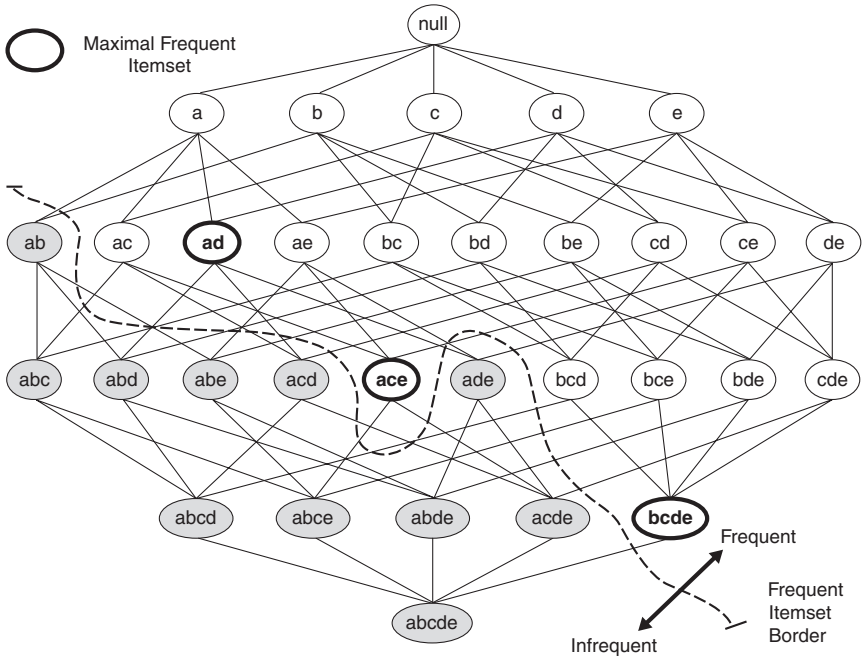


Figure 4.16. Maximal frequent itemset.

the frequent itemsets shown in Figure 4.16. Enumerating all the subsets of maximal frequent itemsets generates the complete list of all frequent itemsets.

Maximal frequent itemsets provide a valuable representation for data sets that can produce very long, frequent itemsets, as there are exponentially many frequent itemsets in such data. Nevertheless, this approach is practical only if an efficient algorithm exists to explicitly find the maximal frequent itemsets. We briefly describe one such approach in Section 4.5.

Despite providing a compact representation, maximal frequent itemsets do not contain the support information of their subsets. For example, the support of the maximal frequent itemsets $\{a, c, e\}$, $\{a, d\}$, and $\{b, c, d, e\}$ do not provide any information about the support of their subsets except that it meets the support threshold. An additional pass over the data set is therefore needed to determine the support counts of the non-maximal frequent itemsets. In some cases, it is desirable to have a minimal representation of itemsets that preserves the support information. We describe such a representation in the next section.

4.4.2 Closed Itemsets

Closed itemsets provide a minimal representation of all itemsets without losing their support information. A formal definition of a closed itemset is presented below.

Definition 4.4 (Closed Itemset)**.** An itemset X is closed if none of its immediate supersets has exactly the same support count as X.

Put another way, X is not closed if at least one of its immediate supersets has the same support count as X. Examples of closed itemsets are shown in Figure 4.17. To better illustrate the support count of each itemset, we have associated each node (itemset) in the lattice with a list of its corresponding transaction IDs. For example, since the node $\{b, c\}$ is associated with transaction IDs 1, 2, and 3, its support count is equal to three. From the transactions given in this diagram, notice that the support for $\{b\}$ is identical to $\{b, c\}$. This is because every transaction that contains b also contains c. Hence, $\{b\}$ is not a closed itemset. Similarly, since c occurs in every transaction that contains both a and d, the itemset $\{a, d\}$ is not closed as it has the same support as its superset $\{a, c, d\}$. On the other hand, $\{b, c\}$ is a closed itemset because it does not have the same support count as any of its supersets.

An interesting property of closed itemsets is that if we know their support counts, we can derive the support count of every other itemset in the itemset lattice without making additional passes over the data set. For example, consider the 2-itemset $\{b, e\}$ in Figure 4.17. Since $\{b, e\}$ is not closed, its support must be equal to the support of one of its immediate supersets, $\{a, b, e\}$, $\{b, c, e\}$, and $\{b, d, e\}$. Further, none of the supersets of $\{b, e\}$ can have a support greater than the support of $\{b, e\}$, due to the anti-monotone nature of the support measure. Hence, the support of $\{b, e\}$ can be computed by examining the support counts of all of its immediate supersets of size three and taking their maximum value. If an immediate superset is closed (e.g., $\{b, c, e\}$), we would know its support count. Otherwise, we can recursively compute its support by examining the supports of its immediate supersets of size four. In general, the support count of any non-closed $(k-1)$-itemset can be determined as long as we know the support counts of all k-itemsets. Hence, one can devise an iterative algorithm that computes the support counts of itemsets at level $k-1$ using the support counts of itemsets at level k, starting from the level $k_{\max}$, where $k_{\max}$ is the size of the largest closed itemset.

Even though closed itemsets provide a compact representation of the support counts of all itemsets, they can still be exponentially large in number. Moreover, for most practical applications, we only need to determine the

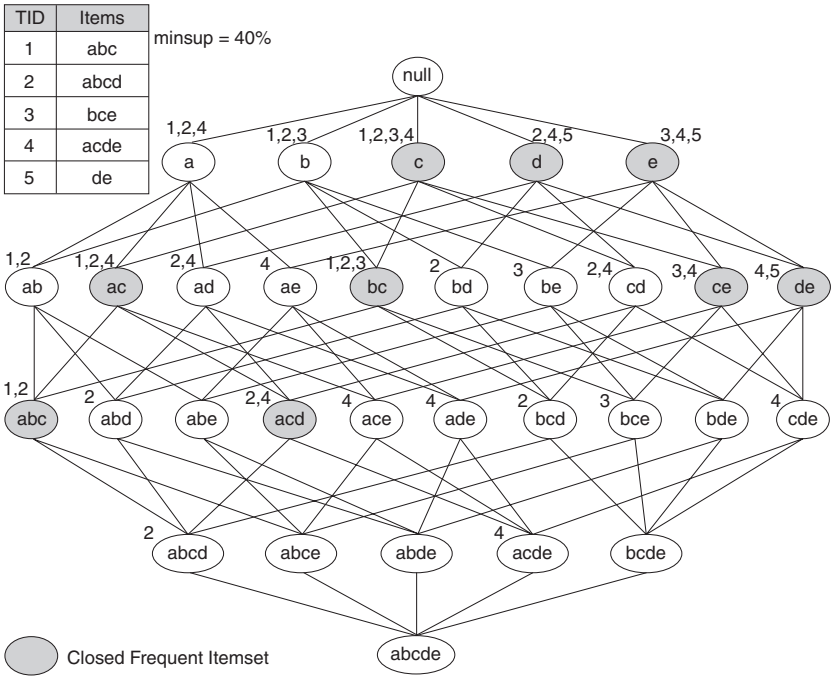


Figure 4.17. An example of the closed frequent itemsets (with minimum support equal to 40%).

support count of all frequent itemsets. In this regard, closed frequent itemsets provide a compact representation of the support counts of all frequent itemsets, which can be defined as follows.

Definition 4.5 (Closed Frequent Itemset)**.** An itemset is a closed frequent itemset if it is closed and its support is greater than or equal to *minsup*.

In the previous example, assuming that the support threshold is 40%, {b,c} is a closed frequent itemset because its support is 60%. In Figure 4.17, the closed frequent itemsets are indicated by the shaded nodes.

Algorithms are available to explicitly extract closed frequent itemsets from a given data set. Interested readers may refer to the Bibliographic Notes at the end of this chapter for further discussions of these algorithms. We can use closed frequent itemsets to determine the support counts for all non-closed frequent itemsets. For example, consider the frequent itemset $\{a, d\}$ shown in Figure 4.17. Because this itemset is not closed, its support count must be equal to the maximum support count of its immediate supersets, $\{a, b, d\}$, $\{a, c, d\}$, and $\{a, d, e\}$. Also, since $\{a, d\}$ is frequent, we only need to consider the support of its frequent supersets. In general, the support count of every

Algorithm 4.4 Support counting using closed frequent itemsets.

1: Let C denote the set of closed frequent itemsets and F denote the set of all frequent itemsets.
2: Let $k_{\max}$ denote the maximum size of closed frequent itemsets
3: $F_{k_{\max}} = \{f | f \in C,\ |f| = k_{\max}\}$ {Find all frequent itemsets of size $k_{\max}$.}
4: **for** $k = k_{\max} - 1$ down to 1 **do**
5: $F_k = \{f | f \in F,\ |f| = k\}$ {Find all frequent itemsets of size k.}
6: **for** each $f \in F_k$ **do**
7: **if** $f \notin C$ **then**
8: $f.support = \max\{f'.support | f' \in F_{k+1},\ f \subset f'\}$
9: **end if**
10: **end for**
11: **end for**

non-closed frequent k-itemset can be obtained by considering the support of all its frequent supersets of size $k + 1$. For example, since the only frequent superset of $\{a, d\}$ is $\{a, c, d\}$, its support is equal to the support of $\{a, c, d\}$, which is 2. Using this methodology, an algorithm can be developed to compute the support for every frequent itemset. The pseudocode for this algorithm is shown in Algorithm 4.4. The algorithm proceeds in a specific-to-general fashion, i.e., from the largest to the smallest frequent itemsets. This is because, in order to find the support for a non-closed frequent itemset, the support for all of its supersets must be known. Note that the set of all frequent itemsets can be easily computed by taking the union of all subsets of frequent closed itemsets.

To illustrate the advantage of using closed frequent itemsets, consider the data set shown in Table 4.5, which contains ten transactions and fifteen items. The items can be divided into three groups: (1) Group A, which contains items a_1 through a_5; (2) Group B, which contains items b_1 through b_5; and (3) Group C, which contains items c_1 through c_5. Assuming that the support threshold is 20%, itemsets involving items from the same group are frequent, but itemsets involving items from different groups are infrequent. The total number of frequent itemsets is thus $3 \times (2^5 - 1) = 93$. However, there are only four closed frequent itemsets in the data: ($\{a_3, a_4\}$, $\{a_1, a_2, a_3, a_4, a_5\}$, $\{b_1, b_2, b_3, b_4, b_5\}$, and $\{c_1, c_2, c_3, c_4, c_5\}$). It is often sufficient to present only the closed frequent itemsets to the analysts instead of the entire set of frequent itemsets.

because it has to generate a large number of subproblems and merge the results returned by each subproblem.

4.7 Evaluation of Association Patterns

Although the *Apriori* principle significantly reduces the exponential search space of candidate itemsets, association analysis algorithms still have the potential to generate a large number of patterns. For example, although the data set shown in Table 4.1 contains only six items, it can produce hundreds of association rules at particular support and confidence thresholds. As the size and dimensionality of real commercial databases can be very large, we can easily end up with thousands or even millions of patterns, many of which might not be interesting. Identifying the most interesting patterns from the multitude of all possible ones is not a trivial task because "one person's trash might be another person's treasure." It is therefore important to establish a set of well-accepted criteria for evaluating the quality of association patterns.

The first set of criteria can be established through a data-driven approach to define **objective interestingness measures**. These measures can be used to rank patterns—itemsets or rules—and thus provide a straightforward way of dealing with the enormous number of patterns that are found in a data set. Some of the measures can also provide statistical information, e.g., itemsets that involve a set of unrelated items or cover very few transactions are considered uninteresting because they may capture spurious relationships in the data and should be eliminated. Examples of objective interestingness measures include support, confidence, and correlation.

The second set of criteria can be established through subjective arguments. A pattern is considered subjectively uninteresting unless it reveals unexpected information about the data or provides useful knowledge that can lead to profitable actions. For example, the rule $\{Butter\} \longrightarrow \{Bread\}$ may not be interesting, despite having high support and confidence values, because the relationship represented by the rule might seem rather obvious. On the other hand, the rule $\{Diapers\} \longrightarrow \{Beer\}$ is interesting because the relationship is quite unexpected and may suggest a new cross-selling opportunity for retailers. Incorporating subjective knowledge into pattern evaluation is a difficult task because it requires a considerable amount of prior information from domain experts. Readers interested in subjective interestingness measures may refer to resources listed in the bibliography at the end of this chapter.

Table 4.6. A 2-way contingency table for variables A and B.

	B	$\overline{B}$	
A	f_{11}	f_{10}	f_{1+}
$\overline{A}$	f_{01}	f_{00}	f_{0+}
	f_{+1}	f_{+0}	N

4.7.1 Objective Measures of Interestingness

An objective measure is a data-driven approach for evaluating the quality of association patterns. It is domain-independent and requires only that the user specifies a threshold for filtering low-quality patterns. An objective measure is usually computed based on the frequency counts tabulated in a **contingency table**. Table 4.6 shows an example of a contingency table for a pair of binary variables, A and B. We use the notation $\overline{A}$ ($\overline{B}$) to indicate that A (B) is absent from a transaction. Each entry f_{ij} in this 2×2 table denotes a frequency count. For example, f_{11} is the number of times A and B appear together in the same transaction, while f_{01} is the number of transactions that contain B but not A. The row sum f_{1+} represents the support count for A, while the column sum f_{+1} represents the support count for B. Finally, even though our discussion focuses mainly on asymmetric binary variables, note that contingency tables are also applicable to other attribute types such as symmetric binary, nominal, and ordinal variables.

Limitations of the Support-Confidence Framework The classical association rule mining formulation relies on the support and confidence measures to eliminate uninteresting patterns. The drawback of support, which is described more fully in Section 4.8, is that many potentially interesting patterns involving low support items might be eliminated by the support threshold. The drawback of confidence is more subtle and is best demonstrated with the following example.

Example 4.3. Suppose we are interested in analyzing the relationship between people who drink tea and coffee. We may gather information about the beverage preferences among a group of people and summarize their responses into a contingency table such as the one shown in Table 4.7.

The information given in this table can be used to evaluate the association rule $\{Tea\} \longrightarrow \{Coffee\}$. At first glance, it may appear that people who drink tea also tend to drink coffee because the rule's support (15%) and confidence

Table 4.7. Beverage preferences among a group of 1000 people.

	$Coffee$	$\overline{Coffee}$	
Tea	150	50	200
$\overline{Tea}$	650	150	800
	800	200	1000

(75%) values are reasonably high. This argument would have been acceptable except that the fraction of people who drink coffee, regardless of whether they drink tea, is 80%, while the fraction of tea drinkers who drink coffee is only 75%. Thus knowing that a person is a tea drinker actually decreases her probability of being a coffee drinker from 80% to 75%! The rule $\{Tea\} \longrightarrow \{Coffee\}$ is therefore misleading despite its high confidence value.

Table 4.8. Information about people who drink tea and people who use honey in their beverage.

	$Honey$	$\overline{Honey}$	
Tea	100	100	200
$\overline{Tea}$	20	780	800
	120	880	1000

Now consider a similar problem where we are interested in analyzing the relationship between people who drink tea and people who use honey in their beverage. Table 4.8 summarizes the information gathered over the same group of people about their preferences for drinking tea and using honey. If we evaluate the association rule $\{Tea\} \longrightarrow \{Honey\}$ using this information, we will find that the confidence value of this rule is merely 50%, which might be easily rejected using a reasonable threshold on the confidence value, say 70%. One thus might consider that the preference of a person for drinking tea has no influence on her preference for using honey. However, the fraction of people who use honey, regardless of whether they drink tea, is only 12%. Hence, knowing that a person drinks tea significantly increases her probability of using honey from 12% to 50%. Further, the fraction of people who do not drink tea but use honey is only 2.5%! This suggests that there is definitely some information in the preference of a person of using honey given that she

drinks tea. The rule $\{Tea\} \longrightarrow \{Honey\}$ may therefore be falsely rejected if confidence is used as the evaluation measure.

■

Note that if we take the support of coffee drinkers into account, we would not be surprised to find that many of the people who drink tea also drink coffee, since the overall number of coffee drinkers is quite large by itself. What is more surprising is that the fraction of tea drinkers who drink coffee is actually less than the overall fraction of people who drink coffee, which points to an inverse relationship between tea drinkers and coffee drinkers. Similarly, if we account for the fact that the support of using honey is inherently small, it is easy to understand that the fraction of tea drinkers who use honey will naturally be small. Instead, what is important to measure is the change in the fraction of honey users, given the information that they drink tea.

The limitations of the confidence measure are well-known and can be understood from a statistical perspective as follows. The support of a variable measures the probability of its occurrence, while the support $s(A, B)$ of a pair of a variables A and B measures the probability of the two variables occurring together. Hence, the joint probability $P(A, B)$ can be written as

$$P(A, B) = s(A, B) = \frac{f_{11}}{N}.$$

If we assume A and B are statistically independent, i.e. there is no relationship between the occurrences of A and B, then $P(A, B) = P(A) \times P(B)$. Hence, under the assumption of statistical independence between A and B, the support $s_{\text{indep}}(A, B)$ of A and B can be written as

$$s_{\text{indep}}(A, B) = s(A) \times s(B) \quad \text{or equivalently,} \quad s_{\text{indep}}(A, B) = \frac{f_{1+}}{N} \times \frac{f_{+1}}{N}. \tag{4.4}$$

If the support between two variables, $s(A, B)$ is equal to $s_{\text{indep}}(A, B)$, then A and B can be considered to be unrelated to each other. However, if $s(A, B)$ is widely different from $s_{\text{indep}}(A, B)$, then A and B are most likely dependent. Hence, any deviation of $s(A, B)$ from $s(A) \times s(B)$ can be seen as an indication of a statistical relationship between A and B. Since the confidence measure only considers the deviance of $s(A, B)$ from $s(A)$ and not from $s(A) \times s(B)$, it fails to account for the support of the consequent, namely $s(B)$. This results in the detection of spurious patterns (e.g., $\{Tea\} \longrightarrow \{Coffee\}$) and the rejection of truly interesting patterns (e.g., $\{Tea\} \longrightarrow \{Honey\}$), as illustrated in the previous example.

Various objective measures have been used to capture the deviance of $s(A, B)$ from $s_{\text{indep}}(A, B)$, that are not susceptible to the limitations of the confidence measure. Below, we provide a brief description of some of these measures and discuss some of their properties.

Interest Factor The interest factor, which is also called as the "lift," can be defined as follows:

$$I(A, B) = \frac{s(A, B)}{s(A) \times s(B)} = \frac{N f_{11}}{f_{1+} f_{+1}}. \tag{4.5}$$

Notice that $s(A) \times s(B) = s_{\text{indep}}(A, B)$. Hence, the interest factor measures the ratio of the support of a pattern $s(A, B)$ against its baseline support $s_{\text{indep}}(A, B)$ computed under the statistical independence assumption. Using Equations 4.5 and 4.4, we can interpret the measure as follows:

$$I(A, B) \begin{cases} = 1, & \text{if } A \text{ and } B \text{ are independent;} \\ > 1, & \text{if } A \text{ and } B \text{ are positively related;} \\ < 1, & \text{if } A \text{ and } B \text{ are negatively related.} \end{cases} \tag{4.6}$$

For the tea-coffee example shown in Table 4.7, $I = \frac{0.15}{0.2 \times 0.8} = 0.9375$, thus suggesting a slight negative relationship between tea drinkers and coffee drinkers. Also, for the tea-honey example shown in Table 4.8, $I = \frac{0.1}{0.12 \times 0.2} = 4.1667$, suggesting a strong positive relationship between people who drink tea and people who use honey in their beverage. We can thus see that the interest factor is able to detect meaningful patterns in the tea-coffee and tea-honey examples. Indeed, the interest factor has a number of statistical advantages over the confidence measure that make it a suitable measure for analyzing statistical independence between variables.

Piatesky-Shapiro (PS) Measure Instead of computing the ratio between $s(A, B)$ and $s_{\text{indep}}(A, B) = s(A) \times s(B)$, the PS measure considers the difference between $s(A, B)$ and $s(A) \times s(B)$ as follows.

$$PS = s(A, B) - s(A) \times s(B) = \frac{f_{11}}{N} - \frac{f_{1+} f_{+1}}{N^2} \tag{4.7}$$

The PS value is 0 when A and B are mutually independent of each other. Otherwise, $PS > 0$ when there is a positive relationship between the two variables, and $PS < 0$ when there is a negative relationship.

Correlation Analysis Correlation analysis is one of the most popular techniques for analyzing relationships between a pair of variables. For continuous variables, correlation is defined using Pearson's correlation coefficient (see Equation 2.10 on page 103). For binary variables, correlation can be measured using the ϕ-coefficient, which is defined as

$$\phi = \frac{f_{11}f_{00} - f_{01}f_{10}}{\sqrt{f_{1+}f_{+1}f_{0+}f_{+0}}}. \tag{4.8}$$

If we rearrange the terms in 4.8, we can show that the ϕ-coefficient can be rewritten in terms of the support measures of A, B, and $\{A, B\}$ as follows:

$$\phi = \frac{s(A,B) - s(A) \times s(B)}{\sqrt{s(A) \times (1 - s(A)) \times s(B) \times (1 - s(B))}}. \tag{4.9}$$

Note that the numerator in the above equation is identical to the PS measure. Hence, the ϕ-coefficient can be understood as a normalized version of the PS measure, where that the value of the ϕ-coefficient ranges from -1 to $+1$. From a statistical viewpoint, the correlation captures the normalized difference between $s(A, B)$ and $s_{\text{indep}}(A, B)$. A correlation value of 0 means no relationship, while a value of $+1$ suggests a perfect positive relationship and a value of -1 suggests a perfect negative relationship. The correlation measure has a statistical meaning and hence is widely used to evaluate the strength of statistical independence among variables. For instance, the correlation between tea and coffee drinkers in Table 4.7 is -0.0625 which is slightly less than 0. On the other hand, the correlation between people who drink tea and people who use honey in Table 4.8 is 0.5847, suggesting a positive relationship.

IS Measure IS is an alternative measure for capturing the relationship between $s(A, B)$ and $s(A) \times s(B)$. The IS measure is defined as follows:

$$IS(A,B) = \sqrt{I(A,B) \times s(A,B)} = \frac{s(A,B)}{\sqrt{s(A)s(B)}} = \frac{f_{11}}{\sqrt{f_{1+}f_{+1}}}. \tag{4.10}$$

Although the definition of IS looks quite similar to the interest factor, they share some interesting differences. Since IS is the geometric mean between the interest factor and the support of a pattern, IS is large when both the interest factor and support are large. Hence, if the interest factor of two patterns are identical, the IS has a preference of selecting the pattern with higher support. It is also possible to show that IS is mathematically equivalent to the cosine

measure for binary variables (see Equation 2.6 on page 101). The value of IS thus varies from 0 to 1, where an IS value of 0 corresponds to no co-occurrence of the two variables, while an IS value of 1 denotes perfect relationship, since they occur in exactly the same transactions. For the tea-coffee example shown in Table 4.7, the value of IS is equal to 0.375, while the value of IS for the tea-honey example in Table 4.8 is 0.6455. The IS measure thus gives a higher value for the $\{Tea\} \longrightarrow \{Honey\}$ rule than the $\{Tea\} \longrightarrow \{Coffee\}$ rule, which is consistent with our understanding of the two rules.

Alternative Objective Interestingness Measures

Note that all of the measures defined in the previous section use different techniques to capture the deviance between $s(A, B)$ and $s_{\text{indep}}(A, B) = s(A) \times s(B)$. Some measures use the ratio between $s(A, B)$ and $s_{\text{indep}}(A, B)$, e.g., the interest factor and IS, while some other measures consider the difference between the two, e.g., the PS and the ϕ-coefficient. Some measures are bounded in a particular range, e.g., the IS and the ϕ-coefficient, while others are unbounded and do not have a defined maximum or minimum value, e.g., the Interest Factor. Because of such differences, these measures behave differently when applied to different types of patterns. Indeed, the measures defined above are not exhaustive and there exist many alternative measures for capturing different properties of relationships between pairs of binary variables. Table 4.9

Table 4.9. Examples of objective measures for the itemset $\{A, B\}$.

Measure (Symbol)	Definition
Correlation (ϕ)	$\frac{Nf_{11} - f_{1+}f_{+1}}{\sqrt{f_{1+}f_{+1}f_{0+}f_{+0}}}$
Odds ratio (α)	$\big(f_{11}f_{00}\big)/\big(f_{10}f_{01}\big)$
Kappa (κ)	$\frac{Nf_{11} + Nf_{00} - f_{1+}f_{+1} - f_{0+}f_{+0}}{N^2 - f_{1+}f_{+1} - f_{0+}f_{+0}}$
Interest (I)	$\big(Nf_{11}\big)/\big(f_{1+}f_{+1}\big)$
Cosine (IS)	$\big(f_{11}\big)/\big(\sqrt{f_{1+}f_{+1}}\big)$
Piatetsky-Shapiro (PS)	$\frac{f_{11}}{N} - \frac{f_{1+}f_{+1}}{N^2}$
Collective strength (S)	$\frac{f_{11}+f_{00}}{f_{1+}f_{+1}+f_{0+}f_{+0}} \times \frac{N-f_{1+}f_{+1}-f_{0+}f_{+0}}{N-f_{11}-f_{00}}$
Jaccard (ζ)	$f_{11}/\big(f_{1+} + f_{+1} - f_{11}\big)$
All-confidence (h)	$\min\big[\frac{f_{11}}{f_{1+}}, \frac{f_{11}}{f_{+1}}\big]$

Table 4.10. Example of contingency tables.

Example	f_{11}	f_{10}	f_{01}	f_{00}
E_1	8123	83	424	1370
E_2	8330	2	622	1046
E_3	3954	3080	5	2961
E_4	2886	1363	1320	4431
E_5	1500	2000	500	6000
E_6	4000	2000	1000	3000
E_7	9481	298	127	94
E_8	4000	2000	2000	2000
E_9	7450	2483	4	63
E_{10}	61	2483	4	7452

provides the definitions for some of these measures in terms of the frequency counts of a 2×2 contingency table.

Consistency among Objective Measures

Given the wide variety of measures available, it is reasonable to question whether the measures can produce similar ordering results when applied to a set of association patterns. If the measures are consistent, then we can choose any one of them as our evaluation metric. Otherwise, it is important to understand what their differences are in order to determine which measure is more suitable for analyzing certain types of patterns.

Suppose the measures defined in Table 4.9 are applied to rank the ten contingency tables shown in Table 4.10. These contingency tables are chosen to illustrate the differences among the existing measures. The ordering produced by these measures is shown in Table 4.11 (with 1 as the most interesting and 10 as the least interesting table). Although some of the measures appear to be consistent with each other, others produce quite different ordering results. For example, the rankings given by the ϕ-coefficient agrees mostly with those provided by κ and collective strength, but are quite different than the rankings produced by interest factor. Furthermore, a contingency table such as E_{10} is ranked lowest according to the ϕ-coefficient, but highest according to interest factor.

Table 4.11. Rankings of contingency tables using the measures given in Table 4.9.

	ϕ	α	κ	I	IS	PS	S	ζ	h
E_1	1	3	1	6	2	2	1	2	2
E_2	2	1	2	7	3	5	2	3	3
E_3	3	2	4	4	5	1	3	6	8
E_4	4	8	3	3	7	3	4	7	5
E_5	5	7	6	2	9	6	6	9	9
E_6	6	9	5	5	6	4	5	5	7
E_7	7	6	7	9	1	8	7	1	1
E_8	8	10	8	8	8	7	8	8	7
E_9	9	4	9	10	4	9	9	4	4
E_{10}	10	5	10	1	10	10	10	10	10

Properties of Objective Measures

The results shown in Table 4.11 suggest that the measures greatly differ from each other and can provide conflicting information about the quality of a pattern. In fact, no measure is universally best for all applications. In the following, we describe some properties of the measures that play an important role in determining if they are suited for a certain application.

Inversion Property Consider the binary vectors shown in Figure 4.28. The 0/1 value in each column vector indicates whether a transaction (row) contains a particular item (column). For example, the vector A indicates that the item appears in the first and last transactions, whereas the vector B indicates that the item is contained only in the fifth transaction. The vectors $\overline{A}$ and $\overline{B}$ are the inverted versions of A and B, i.e., their 1 values have been changed to 0 values (absence to presence) and vice versa. Applying this transformation to a binary vector is called **inversion**. If a measure is invariant under the inversion operation, then its value for the vector pair $\{\overline{A}, \overline{B}\}$ should be identical to its value for $\{A, B\}$. The inversion property of a measure can be tested as follows.

Definition 4.6 (Inversion Property)**.** An objective measure M is invariant under the inversion operation if its value remains the same when exchanging the frequency counts f_{11} with f_{00} and f_{10} with f_{01}.

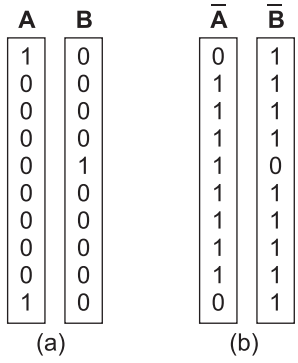


Figure 4.28. Effect of the inversion operation. The vectors $\overline{A}$ and $\overline{E}$ are inversions of vectors A and B, respectively.

Measures that are invariant to the inversion property include the correlation (ϕ-coefficient), odds ratio, κ, and collective strength. These measures are especially useful in scenarios where the presence (1's) of a variable is as important as its absence (0's). For example, if we compare two sets of answers to a series of true/false questions where 0's (true) and 1's (false) are equally meaningful, we should use a measure that gives equal importance to occurrences of 0–0's and 1–1's. For the vectors shown in Figure 4.28, the ϕ-coefficient is equal to -0.1667 regardless of whether we consider the pair $\{A, B\}$ or pair $\{\overline{A}, \overline{B}\}$. Similarly, the odds ratio for both pairs of vectors is equal to a constant value of 0. Note that even though the ϕ-coefficient and the odds ratio are invariant to inversion, they can still show different results, as will be shown later.

Measures that do not remain invariant under the inversion operation include the interest factor and the IS measure. For example, the IS value for the pair $\{\overline{A}, \overline{B}\}$ in Figure 4.28 is 0.825, which reflects the fact that the 1's in $\overline{A}$ and $\overline{B}$ occur frequently together. However, the IS value of its inverted pair $\{A, B\}$ is equal to 0, since A and B do not have any co-occurrence of 1's. For asymmetric binary variables, e.g., the occurrence of words in documents, this is indeed the desired behavior. A desired similarity measure between asymmetric variables should not be invariant to inversion, since for these variables, it is more meaningful to capture relationships based on the presence of a variable rather than its absence. On the other hand, if we are dealing with symmetric binary variables where the relationships between 0's and 1's are equally meaningful, care should be taken to ensure that the chosen measure is invariant to inversion.

Table 4.12. Contingency tables for the pairs $\{p,q\}$ and $\{r,s\}$.

	p	$\overline{p}$	
q	880	50	930
$\overline{q}$	50	20	70
	930	70	1000

	r	$\overline{r}$	
s	20	50	70
$\overline{s}$	50	880	930
	70	930	1000

Although the values of the interest factor and IS change with the inversion operation, they can still be inconsistent with each other. To illustrate this, consider Table 4.12, which shows the contingency tables for two pairs of variables, $\{p, q\}$ and $\{r, s\}$. Note that r and s are inverted transformations of p and q, respectively, where the roles of 0's and 1's have just been reversed. The interest factor for $\{p, q\}$ is 1.02 and for $\{r, s\}$ is 4.08, which means that the interest factor finds the inverted pair $\{r, s\}$ more related than the original pair $\{p, q\}$. On the contrary, the IS value decreases upon inversion from 0.9346 for $\{p, q\}$ to 0.286 for $\{r, s\}$, suggesting quite an opposite trend to that of the interest factor. Even though these measures conflict with each other for this example, they may be the right choice of measure in different applications.

Scaling Property Table 4.13 shows two contingency tables for gender and the grades achieved by students enrolled in a particular course. These tables can be used to study the relationship between gender and performance in the course. The second contingency table has data from the same population but has twice as many males and three times as many females. The actual number of males or females can depend upon the samples available for study, but the relationship between gender and grade should not change just because of differences in sample sizes. Similarly, if the number of students with high and low grades are changed in a new study, a measure of association between gender and grades should remain unchanged. Hence, we need a measure that is invariant to scaling of rows or columns. The process of multiplying a row or column of a contingency table by a constant value is called a row or column scaling operation. A measure that is invariant to scaling does not change its value after any row or column scaling operation.

Definition 4.7 (Scaling Invariance Property)**.** Let T be a contingency table with frequency counts $[f_{11};\ f_{10};\ f_{01};\ f_{00}]$. Let T' be the transformed a contingency table with scaled frequency counts $[k_1k_3f_{11};\ k_2k_3f_{10};\ k_1k_4f_{01};\ k_2k_4f_{00}]$, where k_1, k_2, k_3, k_4 are positive constants used to scale the two rows and the

Table 4.13. The grade-gender example.

	Male	Female	
High	30	20	50
Low	40	10	50
	70	30	100

(a) Sample data of size 100.

	Male	Female	
High	60	60	120
Low	80	30	110
	140	90	230

(b) Sample data of size 230.

two columns of T. An objective measure M is invariant under the row/column scaling operation if $M(T) = M(T')$.

Note that the use of the term 'scaling' here should not be confused with the scaling operation for continuous variables introduced in Chapter 2 on page 43, where all the values of a variable were being multiplied by a constant factor, instead of scaling a row or column of a contingency table.

Scaling of rows and columns in contingency tables occurs in multiple ways in different applications. For example, if we are measuring the effect of a particular medical procedure on two sets of subjects, *healthy* and *diseased*, the ratio of healthy and diseased subjects can widely vary across different studies involving different groups of participants. Further, the fraction of healthy and diseased subjects chosen for a controlled study can be quite different from the true fraction observed in the complete population. These differences can result in a row or column scaling in the contingency tables for different populations of subjects. In general, the frequencies of items in a contingency table closely depends on the sample of transactions used to generate the table. Any change in the sampling procedure may affect a row or column scaling transformation. A measure that is expected to be invariant to differences in the sampling procedure must not change with row or column scaling.

Of all the measures introduced in Table 4.9, only the odds ratio (α) is invariant to row and column scaling operations. For example, the value of odds ratio for both the tables in Table 4.13 is equal to 0.375. All other measures such as the ϕ-coefficient, κ, IS, interest factor, and collective strength (S) change their values when the rows and columns of the contingency table are rescaled. Indeed, the odds ratio is a preferred choice of measure in the medical domain, where it is important to find relationships that do not change with differences in the population sample chosen for a study.

Null Addition Property Suppose we are interested in analyzing the relationship between a pair of words, such as `data` and `mining`, in a set of documents. If a collection of articles about ice fishing is added to the data set, should the association between `data` and `mining` be affected? This process of adding unrelated data (in this case, documents) to a given data set is known as the **null addition** operation.

Definition 4.8 (Null Addition Property)**.** An objective measure M is invariant under the null addition operation if it is not affected by increasing f_{00}, while all other frequencies in the contingency table stay the same.

For applications such as document analysis or market basket analysis, we would like to use a measure that remains invariant under the null addition operation. Otherwise, the relationship between words can be made to change simply by adding enough documents that do not contain both words! Examples of measures that satisfy this property include cosine (IS) and Jaccard (ξ) measures, while those that violate this property include interest factor, PS, odds ratio, and the ϕ-coefficient.

To demonstrate the effect of null addition, consider the two contingency tables T_1 and T_2 shown in Table 4.14. Table T_2 has been obtained from T_1 by adding 1000 extra transactions with both A and B absent. This operation only affects the f_{00} entry of Table T_2, which has increased from 100 to 1100, whereas all the other frequencies in the table (f_{11}, f_{10}, and f_{01}) remain the same. Since IS is invariant to null addition, it gives a constant value of 0.875 to both the tables. However, the addition of 1000 extra transactions with occurrences of 0–0's changes the value of interest factor from 0.972 for T_1 (denoting a slightly negative correlation) to 1.944 for T_2 (positive correlation). Similarly, the value of odds ratio increases from 7 for T_1 to 77 for T_2. Hence, when the interest factor or odds ratio are used as the association measure, the relationships between variables changes by the addition of null transactions where both the variables are absent. In contrast, the IS measure is invariant to null addition, since it considers two variables to be related only if they frequently occur together. Indeed, the IS measure (cosine measure) is widely used to measure similarity among documents, which is expected to depend only on the joint occurrences (1's) of words in documents, but not their absences (0's).

Table 4.15 provides a summary of properties for the measures defined in Table 4.9. Even though this list of properties is not exhaustive, it can serve as a useful guide for selecting the right choice of measure for an application. Ideally, if we know the specific requirements of a certain application, we can ensure that the selected measure shows properties that adhere to those requirements. For example, if we are dealing with asymmetric variables, we would prefer to

Table 4.14. An example demonstrating the effect of null addition.

	B	$\overline{B}$	
A	700	100	800
$\overline{A}$	100	100	200
	800	200	1000

(a) Table T_1.

	B	$\overline{B}$	
A	700	100	800
$\overline{A}$	10	1100	1200
	800	1200	2000

(b) Table T_2.

use a measure that is not invariant to null addition or inversion. On the other hand, if we require the measure to remain invariant to changes in the sample size, we would like to use a measure that does not change with scaling.

Asymmetric Interestingness Measures

Note that in the discussion so far, we have only considered measures that do not change their value when the order of the variables are reversed. More specifically, if M is a measure and A and B are two variables, then $M(A, B)$ is equal to $M(B, A)$ if the order of the variables does not matter. Such measures are called **symmetric**. On the other hand, measures that depend on the order of variables ($M(A, B) \neq M(B, A)$) are called **asymmetric** measures. For example, the interest factor is a symmetric measure because its value is identical for the rules $A \longrightarrow B$ and $B \longrightarrow A$. In contrast, confidence is an asymmetric measure since the confidence for $A \longrightarrow B$ and $B \longrightarrow A$ may not be the same. Note that the use of the term 'asymmetric' to describe a particular type of measure of relationship—one in which the order of the variables is important—should not be confused with the use of 'asymmetric' to describe a binary variable for which only 1's are important. Asymmetric measures are more suitable for analyzing association rules, since the items in a rule do have a specific order. Even though we only considered symmetric measures to discuss the different properties of association measures, the above discussion is also relevant for the asymmetric measures. See Bibliographic Notes for more information about different kinds of asymmetric measures and their properties.

4.7.2 Measures beyond Pairs of Binary Variables

The measures shown in Table 4.9 are defined for pairs of binary variables (e.g., 2-itemsets or association rules). However, many of them, such as support and

Table 4.15. Properties of symmetric measures.

Symbol	Measure	Inversion	Null Addition	Scaling
ϕ	ϕ-coefficient	Yes	No	No
α	odds ratio	Yes	No	Yes
κ	Cohen's	Yes	No	No
I	Interest	No	No	No
IS	Cosine	No	Yes	No
PS	Piatetsky-Shapiro's	Yes	No	No
S	Collective strength	Yes	No	No
ζ	Jaccard	No	Yes	No
h	All-confidence	No	Yes	No
s	Support	No	No	No

Table 4.16. Example of a three-dimensional contingency table.

c	b	$\overline{b}$	
a	f_{111}	f_{101}	f_{1+1}
$\overline{a}$	f_{011}	f_{001}	f_{0+1}
	f_{+11}	f_{+01}	f_{++1}

$\overline{c}$	b	$\overline{b}$	
a	f_{110}	f_{100}	f_{1+0}
$\overline{a}$	f_{010}	f_{000}	f_{0+0}
	f_{+10}	f_{+00}	f_{++0}

all-confidence, are also applicable to larger-sized itemsets. Other measures, such as interest factor, IS, PS, and Jaccard coefficient, can be extended to more than two variables using the frequency tables tabulated in a multidimensional contingency table. An example of a three-dimensional contingency table for a, b, and c is shown in Table 4.16. Each entry f_{ijk} in this table represents the number of transactions that contain a particular combination of items a, b, and c. For example, f_{101} is the number of transactions that contain a and c, but not b. On the other hand, a marginal frequency such as f_{1+1} is the number of transactions that contain a and c, irrespective of whether b is present in the transaction.

Given a k-itemset $\{i_1, i_2, \ldots, i_k\}$, the condition for statistical independence can be stated as follows:

$$f_{i_1 i_2 \ldots i_k} = \frac{f_{i_1+\ldots+} \times f_{+i_2\ldots+} \times \ldots \times f_{++\ldots i_k}}{N^{k-1}}. \tag{4.11}$$

With this definition, we can extend objective measures such as interest factor and PS, which are based on deviations from statistical independence, to more

than two variables:

$$I = \frac{N^{k-1} \times f_{i_1 i_2 \ldots i_k}}{f_{i_1 + \ldots +} \times f_{+ i_2 \ldots +} \times \ldots \times f_{+ + \ldots i_k}}$$

$$PS = \frac{f_{i_1 i_2 \ldots i_k}}{N} - \frac{f_{i_1 + \ldots +} \times f_{+ i_2 \ldots +} \times \ldots \times f_{+ + \ldots i_k}}{N^k}$$

Another approach is to define the objective measure as the maximum, minimum, or average value for the associations between pairs of items in a pattern. For example, given a k-itemset $X = \{i_1, i_2, \ldots, i_k\}$, we may define the ϕ-coefficient for X as the average ϕ-coefficient between every pair of items (i_p, i_q) in X. However, because the measure considers only pairwise associations, it may not capture all the underlying relationships within a pattern. Also, care should be taken in using such alternate measures for more than two variables, since they may not always show the anti-monotone property in the same way as the support measure, making them unsuitable for mining patterns using the *Apriori* principle.

Analysis of multidimensional contingency tables is more complicated because of the presence of partial associations in the data. For example, some associations may appear or disappear when conditioned upon the value of certain variables. This problem is known as **Simpson's paradox** and is described in Section 4.7.3. More sophisticated statistical techniques are available to analyze such relationships, e.g., loglinear models, but these techniques are beyond the scope of this book.

4.7.3 Simpson's Paradox

It is important to exercise caution when interpreting the association between variables because the observed relationship may be influenced by the presence of other confounding factors, i.e., hidden variables that are not included in the analysis. In some cases, the hidden variables may cause the observed relationship between a pair of variables to disappear or reverse its direction, a phenomenon that is known as Simpson's paradox. We illustrate the nature of this paradox with the following example.

Consider the relationship between the sale of high-definition televisions (HDTV) and exercise machines, as shown in Table 4.17. The rule {HDTV=Yes} $\longrightarrow$ {Exercise machine=Yes} has a confidence of $99/180 = 55\%$ and the rule {HDTV=No} $\longrightarrow$ {Exercise machine=Yes} has a confidence of $54/120 = 45\%$. Together, these rules suggest that customers who buy high-definition televisions are more likely to buy exercise machines than those who do not buy high-definition televisions.

Table 4.17. A two-way contingency table between the sale of high-definition television and exercise machine.

Buy HDTV	Buy Exercise Machine		
	Yes	No	
Yes	99	81	180
No	54	66	120
	153	147	300

Table 4.18. Example of a three-way contingency table.

Customer Group	Buy HDTV	Buy Exercise Machine		Total
		Yes	No	
College Students	Yes	1	9	10
	No	4	30	34
Working Adult	Yes	98	72	170
	No	50	36	86

However, a deeper analysis reveals that the sales of these items depend on whether the customer is a college student or a working adult. Table 4.18 summarizes the relationship between the sale of HDTVs and exercise machines among college students and working adults. Notice that the support counts given in the table for college students and working adults sum up to the frequencies shown in Table 4.17. Furthermore, there are more working adults than college students who buy these items. For college students:

$$c\big(\{\text{HDTV=Yes}\} \longrightarrow \{\text{Exercise machine=Yes}\}\big) = 1/10 = 10\%,$$
$$c\big(\{\text{HDTV=No}\} \longrightarrow \{\text{Exercise machine=Yes}\}\big) = 4/34 = 11.8\%,$$

while for working adults:

$$c\big(\{\text{HDTV=Yes}\} \longrightarrow \{\text{Exercise machine=Yes}\}\big) = 98/170 = 57.7\%,$$
$$c\big(\{\text{HDTV=No}\} \longrightarrow \{\text{Exercise machine=Yes}\}\big) = 50/86 = 58.1\%.$$

The rules suggest that, for each group, customers who do not buy high-definition televisions are more likely to buy exercise machines, which contradicts the previous conclusion when data from the two customer groups are pooled together. Even if alternative measures such as correlation, odds ratio, or interest are applied, we still find that the sale of HDTV and exercise machine is positively related in the combined data but is negatively related in

7

Association Analysis: Advanced Concepts

The association rule mining formulation described in Chapter 4 assumes that the input data consists of binary attributes called items. The presence of an item in a transaction is also assumed to be more important than its absence. As a result, an item is treated as an asymmetric binary attribute and only frequent patterns are considered interesting.

This chapter extends the formulation to data sets with symmetric binary, categorical, and continuous attributes. The formulation will also be extended to incorporate more complex entities such as sequences and graphs. Although the overall structure of association analysis algorithms remains unchanged, certain aspects of the algorithms must be modified to handle the non-traditional entities.

7.1 Handling Categorical Attributes

There are many applications that contain symmetric binary and nominal attributes. The Internet survey data shown in Table 7.1 contains symmetric binary attributes such as `Gender`, `Computer at Home`, `Chat Online`, `Shop Online`, and `Privacy Concerns`; as well as nominal attributes such as `Level of Education` and `State`. Using association analysis, we may uncover interesting information about the characteristics of Internet users such as

$$\{\texttt{Shop Online = Yes}\} \longrightarrow \{\texttt{Privacy Concerns = Yes}\}.$$

This rule suggests that most Internet users who shop online are concerned about their personal privacy.

Table 7.1. Internet survey data with categorical attributes.

Gender	Level of Education	State	Computer at Home	Chat Online	Shop Online	Privacy Concerns
Female	Graduate	Illinois	Yes	Yes	Yes	Yes
Male	College	California	No	No	No	No
Male	Graduate	Michigan	Yes	Yes	Yes	Yes
Female	College	Virginia	No	No	Yes	Yes
Female	Graduate	California	Yes	No	No	Yes
Male	College	Minnesota	Yes	Yes	Yes	Yes
Male	College	Alaska	Yes	Yes	Yes	No
Male	High School	Oregon	Yes	No	No	No
Female	Graduate	Texas	No	Yes	No	No
...	...	...	...	...	...	...

To extract such patterns, the categorical and symmetric binary attributes are transformed into "items" first, so that existing association rule mining algorithms can be applied. This type of transformation can be performed by creating a new item for each distinct attribute-value pair. For example, the nominal attribute `Level of Education` can be replaced by three binary items: `Education = College`, `Education = Graduate`, and `Education = High School`. Similarly, symmetric binary attributes such as `Gender` can be converted into a pair of binary items, `Male` and `Female`. Table 7.2 shows the result of binarizing the Internet survey data.

Table 7.2. Internet survey data after binarizing categorical and symmetric binary attributes.

Male	Female	Education = Graduate	Education = College	...	Privacy = Yes	Privacy = No
0	1	1	0	...	1	0
1	0	0	1	...	0	1
1	0	1	0	...	1	0
0	1	0	1	...	1	0
0	1	1	0	...	1	0
1	0	0	1	...	1	0
1	0	0	1	...	0	1
1	0	0	0	...	0	1
0	1	1	0	...	0	1
...	...	...	...	...	...	...

There are several issues to consider when applying association analysis to the binarized data:

1. Some attribute values may not be frequent enough to be part of a frequent pattern. This problem is more evident for nominal attributes that have many possible values, e.g., state names. Lowering the support threshold does not help because it exponentially increases the number of frequent patterns found (many of which may be spurious) and makes the computation more expensive. A more practical solution is to group related attribute values into a small number of categories. For example, each state name can be replaced by its corresponding geographical region, such as `Midwest`, `Pacific Northwest`, `Southwest`, and `East Coast`. Another possibility is to aggregate the less frequent attribute values into a single category called `Others`, as shown in Figure 7.1.

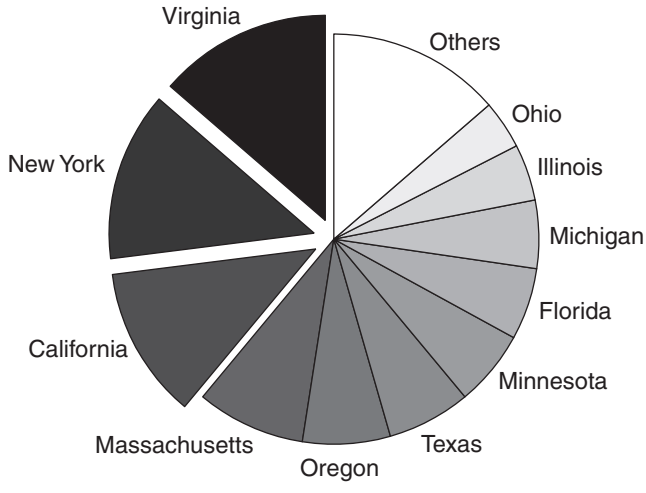


Figure 7.1. A pie chart with a merged category called `Others`.

2. Some attribute values may have considerably higher frequencies than others. For example, suppose 85% of the survey participants own a home computer. By creating a binary item for each attribute value that appears frequently in the data, we may potentially generate many redundant patterns, as illustrated by the following example:

$$\{\texttt{Computer at home = Yes}, \texttt{Shop Online = Yes}\} \longrightarrow \{\texttt{Privacy Concerns = Yes}\}.$$

The rule is redundant because it is subsumed by the more general rule given at the beginning of this section. Because the high-frequency items correspond to the typical values of an attribute, they seldom carry any new information that can help us to better understand the pattern. It may therefore be useful to remove such items before applying standard association analysis algorithms. Another possibility is to apply the techniques presented in Section 4.8 for handling data sets with a wide range of support values.

3. Although the width of every transaction is the same as the number of attributes in the original data, the computation time may increase especially when many of the newly created items become frequent. This is because more time is needed to deal with the additional candidate itemsets generated by these items (see Exercise 1 on page 618). One way to reduce the computation time is to avoid generating candidate itemsets that contain more than one item from the same attribute. For example, we do not have to generate a candidate itemset such as {`State = X`, `State = Y`, ...} because the support count of the itemset is zero.

7.2 Handling Continuous Attributes

The Internet survey data described in the previous section may also contain continuous attributes such as the ones shown in Table 7.3. Mining the continuous attributes may reveal useful insights about the data such as "users whose annual income is more than $120K belong to the 45–60 age group" or "users who have more than 3 email accounts and spend more than 15 hours online per week are often concerned about their personal privacy." Association rules that contain continuous attributes are commonly known as **quantitative association rules**.

This section describes the various methodologies for applying association analysis to continuous data. We will specifically discuss three types of methods: (1) discretization-based methods, (2) statistics-based methods, and (3) non-discretization methods. The quantitative association rules derived using these methods are quite different in nature.

7.2.1 Discretization-Based Methods

Discretization is the most common approach for handling continuous attributes. This approach groups the adjacent values of a continuous attribute into a finite number of intervals. For example, the `Age` attribute can be divided into

Table 7.3. Internet survey data with continuous attributes.

Gender	...	Age	Annual Income	No. of Hours Spent Online per Week	No. of Email Accounts	Privacy Concern
Female	...	26	90K	20	4	Yes
Male	...	51	135K	10	2	No
Male	...	29	80K	10	3	Yes
Female	...	45	120K	15	3	Yes
Female	...	31	95K	20	5	Yes
Male	...	25	55K	25	5	Yes
Male	...	37	100K	10	1	No
Male	...	41	65K	8	2	No
Female	...	26	85K	12	1	No
...	...	...	...	...	...	...

the following intervals: $\texttt{Age} \in [12, 16)$, $\texttt{Age} \in [16, 20)$, $\texttt{Age} \in [20, 24), \ldots,$ $\texttt{Age} \in [56, 60)$, where $[a, b)$ represents an interval that includes a but not b. Discretization can be performed using any of the techniques described in Section 2.3.6 (equal interval width, equal frequency, entropy-based, or clustering). The discrete intervals are then mapped into asymmetric binary attributes so that existing association analysis algorithms can be applied. Table 7.4 shows the Internet survey data after discretization and binarization.

Table 7.4. Internet survey data after binarizing categorical and continuous attributes.

Male	Female	...	Age < 13	Age $\in [13, 21)$	Age $\in [21, 30)$	...	Privacy = Yes	Privacy = No
0	1	...	0	0	1	...	1	0
1	0	...	0	0	0	...	0	1
1	0	...	0	0	1	...	1	0
0	1	...	0	0	0	...	1	0
0	1	...	0	0	0	...	1	0
1	0	...	0	0	1	...	1	0
1	0	...	0	0	0	...	0	1
1	0	...	0	0	0	...	0	1
0	1	...	0	0	1	...	0	1
...	...	...	...	...	...	...	...	...

A key parameter in attribute discretization is the number of intervals used to partition each attribute. This parameter is typically provided by the users and can be expressed in terms of the interval width (for the equal interval

Table 7.5. A breakdown of Internet users who participated in online chat according to their age group.

Age Group	Chat Online = Yes	Chat Online = No
[12, 16)	12	13
[16, 20)	11	2
[20, 24)	11	3
[24, 28)	12	13
[28, 32)	14	12
[32, 36)	15	12
[36, 40)	16	14
[40, 44)	16	14
[44, 48)	4	10
[48, 52)	5	11
[52, 56)	5	10
[56, 60)	4	11

width approach), the average number of transactions per interval (for the equal frequency approach), or the number of desired clusters (for the clustering-based approach). The difficulty in determining the right number of intervals can be illustrated using the data set shown in Table 7.5, which summarizes the responses of 250 users who participated in the survey. There are two strong rules embedded in the data:

R_1: Age $\in [16, 24) \longrightarrow$ Chat Online = Yes (s = 8.8%, c = 81.5%).
R_2: Age $\in [44, 60) \longrightarrow$ Chat Online = No (s = 16.8%, c = 70%).

These rules suggest that most of the users from the age group of 16–24 often participate in online chatting, while those from the age group of 44–60 are less likely to chat online. In this example, we consider a rule to be interesting only if its support (s) exceeds 5% and its confidence (c) exceeds 65%. One of the problems encountered when discretizing the `Age` attribute is how to determine the interval width.

1. If the interval is too wide, then we may lose some patterns because of their lack of confidence. For example, when the interval width is 24 years, R_1 and R_2 are replaced by the following rules:

   R'_1: Age $\in [12, 36) \longrightarrow$ Chat Online = Yes (s = 30%, c = 57.7%).
   R'_2: Age $\in [36, 60) \longrightarrow$ Chat Online = No (s = 28%, c = 58.3%).

   Despite their higher supports, the wider intervals have caused the confidence for both rules to drop below the minimum confidence threshold. As a result, both patterns are lost after discretization.

2. If the interval is too narrow, then we may lose some patterns because of their lack of support. For example, if the interval width is 4 years, then R_1 is broken up into the following two subrules:

$$R_{11}^{(4)}: \quad \text{Age} \in [16, 20) \longrightarrow \text{Chat Online} = \text{Yes (s=4.4\%, c=84.6\%)}.$$
$$R_{12}^{(4)}: \quad \text{Age} \in [20, 24) \longrightarrow \text{Chat Online} = \text{Yes (s=4.4\%, c=78.6\%)}.$$

Since the supports for the subrules are less than the minimum support threshold, R_1 is lost after discretization. Similarly, the rule R_2, which is broken up into four subrules, will also be lost because the support of each subrule is less than the minimum support threshold.

3. If the interval width is 8 years, then the rule R_2 is broken up into the following two subrules:

$$R_{21}^{(8)}: \quad \text{Age} \in [44, 52) \longrightarrow \text{Chat Online} = \text{No (s=8.4\%, c=70\%)}.$$
$$R_{22}^{(8)}: \quad \text{Age} \in [52, 60) \longrightarrow \text{Chat Online} = \text{No (s=8.4\%, c=70\%)}.$$

Since $R_{21}^{(8)}$ and $R_{22}^{(8)}$ have sufficient support and confidence, R_2 can be recovered by aggregating both subrules. Meanwhile, R_1 is broken up into the following two subrules:

$$R_{11}^{(8)}: \quad \text{Age} \in [12, 20) \longrightarrow \text{Chat Online} = \text{Yes (s=9.2\%, c=60.5\%)}.$$
$$R_{12}^{(8)}: \quad \text{Age} \in [20, 28) \longrightarrow \text{Chat Online} = \text{Yes (s=9.2\%, c=60.0\%)}.$$

Unlike R_2, we cannot recover the rule R_1 by aggregating the subrules because both subrules fail the confidence threshold.

One way to address these issues is to consider every possible grouping of adjacent intervals. For example, we can start with an interval width of 4 years and then merge the adjacent intervals into wider intervals: `Age` $\in [12, 16)$, `Age` $\in [12, 20)$, $\ldots$, `Age` $\in [12, 60)$, `Age` $\in [16, 20)$, `Age` $\in [16, 24)$, etc. This approach enables the detection of both R_1 and R_2 as strong rules. However, it also leads to the following computational issues:

1. **The computation becomes extremely expensive**. If the range is initially divided into k intervals, then $k(k-1)/2$ binary items must be generated to represent all possible intervals. Furthermore, if an item corresponding to the interval [a,b) is frequent, then all other items corresponding to intervals that subsume [a,b) must be frequent too. This

approach can therefore generate far too many candidate and frequent itemsets. To address these problems, a maximum support threshold can be applied to prevent the creation of items corresponding to very wide intervals and to reduce the number of itemsets.

2. **Many redundant rules are extracted**. For example, consider the following pair of rules:

   R_3 : {Age $\in [16, 20)$, Gender = Male} $\longrightarrow$ {Chat Online = Yes},

   R_4 : {Age $\in [16, 24)$, Gender = Male} $\longrightarrow$ {Chat Online = Yes}.

   R_4 is a generalization of R_3 (and R_3 is a specialization of R_4) because R_4 has a wider interval for the Age attribute. If the confidence values for both rules are the same, then R_4 should be more interesting because it covers more examples—including those for R_3. R_3 is therefore a redundant rule.

7.2.2 Statistics-Based Methods

Quantitative association rules can be used to infer the statistical properties of a population. For example, suppose we are interested in finding the average age of certain groups of Internet users based on the data provided in Tables 7.1 and 7.3. Using the statistics-based method described in this section, quantitative association rules such as the following can be extracted:

$$\{\texttt{Annual Income} > \$100\text{K}, \texttt{Shop Online = Yes}\} \longrightarrow \texttt{Age: Mean} = 38.$$

The rule states that the average age of Internet users whose annual income exceeds \$100K and who shop online regularly is 38 years old.

Rule Generation

To generate the statistics-based quantitative association rules, the target attribute used to characterize interesting segments of the population must be specified. By withholding the target attribute, the remaining categorical and continuous attributes in the data are binarized using the methods described in the previous section. Existing algorithms such as *Apriori* or FP-growth are then applied to extract frequent itemsets from the binarized data. Each frequent itemset identifies an interesting segment of the population. The distribution of the target attribute in each segment can be summarized using descriptive statistics such as mean, median, variance, or absolute deviation. For example, the preceding rule is obtained by averaging the age of Internet

users who support the frequent itemset {`Annual Income` > \$100K, `Shop Online = Yes`}.

The number of quantitative association rules discovered using this method is the same as the number of extracted frequent itemsets. Because of the way the quantitative association rules are defined, the notion of confidence is not applicable to such rules. An alternative method for validating the quantitative association rules is presented next.

Rule Validation

A quantitative association rule is interesting only if the statistics computed from transactions covered by the rule are different than those computed from transactions not covered by the rule. For example, the rule given at the beginning of this section is interesting only if the average age of Internet users who do not support the frequent itemset {`Annual Income` > 100K, `Shop Online = Yes`} is significantly higher or lower than 38 years old. To determine whether the difference in their average ages is statistically significant, statistical hypothesis testing methods should be applied.

Consider the quantitative association rule, $A \longrightarrow t : \mu$, where A is a frequent itemset, t is the continuous target attribute, and μ is the average value of t among transactions covered by A. Furthermore, let μ' denote the average value of t among transactions not covered by A. The goal is to test whether the difference between μ and μ' is greater than some user-specified threshold, Δ. In statistical hypothesis testing, two opposite propositions, known as the null hypothesis and the alternative hypothesis, are given. A hypothesis test is performed to determine which of these two hypotheses should be accepted, based on evidence gathered from the data (see Appendix C).

In this case, assuming that $\mu < \mu'$, the null hypothesis is $H_0 : \mu' = \mu + \Delta$, while the alternative hypothesis is $H_1 : \mu' > \mu + \Delta$. To determine which hypothesis should be accepted, the following Z-statistic is computed:

$$Z = \frac{\mu' - \mu - \Delta}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}, \tag{7.1}$$

where n_1 is the number of transactions supporting A, n_2 is the number of transactions not supporting A, s_1 is the standard deviation for t among transactions that support A, and s_2 is the standard deviation for t among transactions that do not support A. Under the null hypothesis, Z has a standard normal distribution with mean 0 and variance 1. The value of Z computed using Equation 7.1 is then compared against a critical value, Z_α,

which is a threshold that depends on the desired confidence level. If $Z > Z_\alpha$, then the null hypothesis is rejected and we may conclude that the quantitative association rule is interesting. Otherwise, there is not enough evidence in the data to show that the difference in mean is statistically significant.

Example 7.1. Consider the quantitative association rule

$$\{Income > 100K,\ Shop\ Online = Yes\} \longrightarrow Age : \mu = 38.$$

Suppose there are 50 Internet users who supported the rule antecedent. The standard deviation of their ages is 3.5. On the other hand, the average age of the 200 users who do not support the rule antecedent is 30 and their standard deviation is 6.5. Assume that a quantitative association rule is considered interesting only if the difference between μ and μ' is more than 5 years. Using Equation 7.1 we obtain

$$Z = \frac{38 - 30 - 5}{\sqrt{\frac{3.5^2}{50} + \frac{6.5^2}{200}}} = 4.4414.$$

For a one-sided hypothesis test at a 95% confidence level, the critical value for rejecting the null hypothesis is 1.64. Since $Z > 1.64$, the null hypothesis can be rejected. We therefore conclude that the quantitative association rule is interesting because the difference between the average ages of users who support and do not support the rule antecedent is more than 5 years. ■

7.2.3 Non-discretization Methods

There are certain applications in which analysts are more interested in finding associations among the continuous attributes, rather than associations among discrete intervals of the continuous attributes. For example, consider the problem of finding word associations in text documents. Table 7.6 shows a document-word matrix where every entry represents the number of times a word appears in a given document. Given such a data matrix, we are interested in finding associations between words (e.g., `data` and `mining`) instead of associations between ranges of word counts (e.g., `data` $\in [1, 4]$ and `mining` $\in [2, 3]$). One way to do this is to transform the data into a 0/1 matrix, where the entry is 1 if the count exceeds some threshold t, and 0 otherwise. While this approach allows analysts to apply existing frequent itemset generation algorithms to the binarized data set, finding the right threshold for binarization can be quite tricky. If the threshold is set too high, it is possible to miss some interesting

Table 7.6. Document-word matrix.

Document	$word_1$	$word_2$	$word_3$	$word_4$	$word_5$	$word_6$
d_1	3	6	0	0	0	2
d_2	1	2	0	0	0	2
d_3	4	2	7	0	0	2
d_4	2	0	3	0	0	1
d_5	0	0	0	1	1	0

associations. Conversely, if the threshold is set too low, there is a potential for generating a large number of spurious associations.

This section presents another methodology for finding associations among continuous attributes, known as the min-*Apriori* approach. Analogous to traditional association analysis, an itemset is considered to be a collection of continuous attributes, while its support measures the degree of association among the attributes, across multiple rows of the data matrix. For example, a collection of words in Table 7.6 can be referred to as an itemset, whose support is determined by the co-occurrence of words across documents. In min-*Apriori*, the association among attributes in a given row of the data matrix is obtained by taking the minimum value of the attributes. For example, the association between words, $word_1$ and $word_2$, in the document d_1 is given by $\min(3, 6) = 3$. The support of an itemset is then computed by aggregating its association over all the documents.

$$\begin{aligned} s(\{word_1, word_2\}) &= \min(3,6) + \min(1,2) + \min(4,2) \\ &\quad + \min(2,0) \\ &= 6. \end{aligned}$$

The support measure defined in min-*Apriori* has the following desired properties, which makes it suitable for finding word associations in documents:

1. Support increases monotonically as the number of occurrences of a word increases.

2. Support increases monotonically as the number of documents that contain the word increases.

3. Support has an anti-monotone property. For example, consider a pair of itemsets $\{A, B\}$ and $\{A, B, C\}$. Since $\min(\{A, B\}) \geq \min(\{A, B, C\})$, $s(\{A, B\}) \geq s(\{A, B, C\})$. Therefore, support decreases monotonically as the number of words in an itemset increases.

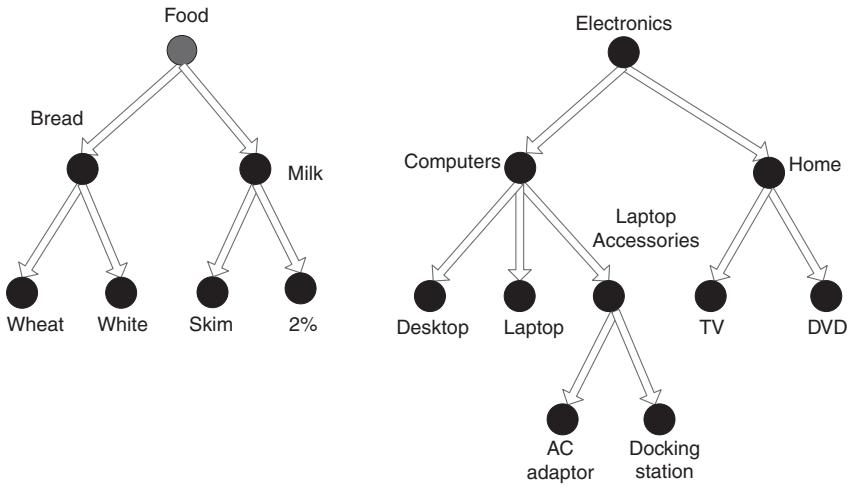


Figure 7.2. Example of an item taxonomy.

The standard *Apriori* algorithm can be modified to find associations among words using the new support definition.

7.3 Handling a Concept Hierarchy

A concept hierarchy is a multilevel organization of the various entities or concepts defined in a particular domain. For example, in market basket analysis, a concept hierarchy has the form of an item taxonomy describing the "is-a" relationships among items sold at a grocery store—e.g., milk is a kind of food and DVD is a kind of home electronics equipment (see Figure 7.2). Concept hierarchies are often defined according to domain knowledge or based on a standard classification scheme defined by certain organizations (e.g., the Library of Congress classification scheme is used to organize library materials based on their subject categories).

A concept hierarchy can be represented using a **directed acyclic graph**, as shown in Figure 7.2. If there is an edge in the graph from a node p to another node c, we call p the **parent** of c and c the **child** of p. For example, `milk` is the parent of `skim milk` because there is a directed edge from the node `milk` to the node `skim milk`. $\hat{X}$ is called an **ancestor** of X (and X a **descendent** of $\hat{X}$) if there is a path from node $\hat{X}$ to node X in the directed acyclic graph. In the diagram shown in Figure 7.2, `food` is an ancestor of `skim milk` and `AC adaptor` is a descendent of `electronics`.

The main advantages of incorporating concept hierarchies into association analysis are as follows:

1. Items at the lower levels of a hierarchy may not have enough support to appear in any frequent itemset. For example, although the sale of AC adaptors and docking stations may be low, the sale of laptop accessories, which is their parent node in the concept hierarchy, may be high. Also, rules involving high-level categories may have lower confidence than the ones generated using low-level categories. Unless the concept hierarchy is used, there is a potential to miss interesting patterns at different levels of categories.

2. Rules found at the lower levels of a concept hierarchy tend to be overly specific and may not be as interesting as rules at the higher levels. For example, staple items such as milk and bread tend to produce many low-level rules such as `skim milk` $\longrightarrow$ `wheat bread`, `2% milk` $\longrightarrow$ `wheat bread`, and `skim milk` $\longrightarrow$ `white bread`. Using a concept hierarchy, they can be summarized into a single rule, `milk` $\longrightarrow$ `bread`. Considering only items residing at the top level of their hierarchies also may not be good enough, because such rules may not be of any practical use. For example, although the rule `electronics` $\longrightarrow$ `food` may satisfy the support and confidence thresholds, it is not informative because the combination of electronics and food items that are frequently purchased by customers are unknown. If milk and batteries are the only items sold together frequently, then the pattern {`food`, `electronics`} may have overgeneralized the situation.

Standard association analysis can be extended to incorporate concept hierarchies in the following way. Each transaction t is initially replaced with its **extended transaction** t', which contains all the items in t along with their corresponding ancestors. For example, the transaction {`DVD`, `wheat bread`} can be extended to {`DVD`, `wheat bread`, `home electronics`, `electronics`, `bread`, `food`}, where `home electronics` and `electronics` are the ancestors of `DVD`, while `bread` and `food` are the ancestors of `wheat bread`.

We can then apply existing algorithms, such as *Apriori*, to the database of extended transactions. Although such an approach would find rules that span different levels of the concept hierarchy, it would suffer from several obvious limitations as described below:

1. Items residing at the higher levels tend to have higher support counts than those residing at the lower levels of a concept hierarchy. As a result,